

NBSAP 2025–2030 Biodiversity Monitoring Dashboard

System Analysis, Requirements Specification, Business Process, and Architecture Document

Prepared for: Rwanda Environment Management Authority (REMA), Ministry of Environment (MoE), MINECOFIN, Project Reviewers, Auditors, and Stakeholders

Document Reference: NBSAP-COMP-2026-001

Version: 1.0

Date: June 2026

Classification: Official — Government Use

System URL: <https://nbsap-dashboard-rw.vercel.app>

Document Control

Field	Value
Document Title	NBSAP 2025–2030 System Analysis, Requirements, Business Process & Architecture Document
Project	Rwanda NBSAP Biodiversity Monitoring Dashboard
Prepared by	NBSAP Dashboard Development Team
Approved by	REMA — Department of Biodiversity & Landscape Management
Status	Final
Total Pages	Comprehensive (20 Sections)

TABLE OF CONTENTS

- 1. Executive Summary
- 2. System Overview

3. Business Process Document
 4. Software Requirements Specification (SRS)
 5. User Roles and Permissions
 6. Business Rules
 7. Data Requirements
 8. Use Case Specification
 9. System Behaviour
 10. Validation Rules
 11. Error Handling
 12. System Architecture
 13. Component Diagram
 14. Data Flow Diagram (DFD)
 15. Entity Relationship Diagram (ERD)
 16. Database Architecture
 17. Security Architecture
 18. Integrations
 19. Deployment Architecture
 20. Appendices
-

SECTION 1: EXECUTIVE SUMMARY

1.1 Project Overview

The **Rwanda NBSAP Monitoring Dashboard** is a national digital platform developed for the Rwanda Environment Management Authority (REMA) to track, report, verify, and visualise Rwanda's progress toward its 22 National Biodiversity Strategy and Action Plan (NBSAP) 2025–2030 targets. These targets are aligned with the 23 global targets of the Kunming-Montreal Global Biodiversity Framework (KM-GBF) adopted at COP15 in December 2022.

The system replaces the prior fragmented approach of manual spreadsheets, PDF reports, and email-based review cycles with a single, authoritative, real-time digital platform that serves as Rwanda's national biodiversity monitoring infrastructure.

1.2 System Purpose

The NBSAP Monitoring Dashboard exists to:

1. **Digitise biodiversity data collection** — replacing paper-based and email-driven reporting with 7 structured digital reporting tools (T01–T07)
2. **Enforce data quality** — implementing a mandatory verification queue where every submission must be reviewed and approved before it affects national metrics

3. **Automate metric computation** — using PostgreSQL database triggers to calculate 13 system-wide metrics from approved reports, eliminating human computation error
4. **Provide real-time national visibility** — delivering live dashboards, interactive maps, and progress indicators to all stakeholders from district officers to Cabinet-level decision-makers
5. **Support CBD reporting obligations** — structuring indicator data according to the KM-GBF three-tier framework for direct export in Rwanda's periodic National Reports to the CBD Secretariat

1.3 Business Objectives

Objective	Measurable Outcome
Eliminate manual data compilation	Real-time automated metrics replace quarterly manual aggregation
Enforce data integrity	100% of submissions pass through mandatory verification queue
Ensure audit trail compliance	Every system action logged immutably with user identity and timestamp
Enable multi-stakeholder access	6 distinct user roles with scoped permissions serving ~200 users
Support CBD obligations	Indicator data exportable in CBD-compliant 3-tier structure (CSV, PDF, JSON)
Reduce reporting turnaround	Average verification time target: < 48 hours (down from weeks)
Achieve national geographic coverage	30 districts across 5 provinces with interactive map visualisation

1.4 Alignment with Rwanda NBSAP 2025–2030

The system directly implements monitoring infrastructure for all 22 national targets:

KM-GBF Goal	Rwanda Targets	Focus Area
Goal A	Targets 1–8	Reduce threats: spatial planning, ecosystem restoration, protected areas, species conservation, IAS, pollution, climate resilience
Goal B	Targets 9–12	Meet people's needs: community benefits, sustainable agriculture, ecosystem services, urban green spaces
Goal C	Targets 13–17	Tools & solutions: ABS, mainstreaming, business disclosure, SCP, biosafety
Goal D	Targets 18–22	Implementation: incentive reform, finance mobilisation (\$500M), capacity building, data access, inclusive participation

1.5 Key Stakeholders

Stakeholder	Category	Interaction with System
REMA	System Owner	Full administration, verification, CBD reporting, user management
Ministry of Environment (MoE)	Policy Oversight	Strategic dashboard monitoring, policy alignment
MINECOFIN	Financial Oversight	Budget and finance tracking via T05 reports
Lead Government Ministries (MINAGRI, MININFRA, etc.)	Data Providers	Submit T01–T07 reports, review submissions
Rwanda Development Board (RDB)	Conservation Partner	Protected area data (T03), tourism revenue sharing
Rwanda Forestry Authority (RFA)	Conservation Partner	Forest and ecosystem data (T02)
District Local Governments (30 districts)	Field Operations	District-level data entry and monitoring
Development Partners (UNDP, GIZ, USAID)	Analytical Users	Programme alignment, donor reporting
University of Rwanda / RBIS	Data Integration	Biodiversity data streams, research
CBD Secretariat	International Body	Receives structured national reports
General Public	Transparency Audience	View-only access to dashboards and maps

1.6 Expected Benefits

Benefit Category	Before (AS-IS)	After (TO-BE)
Data compilation	Weeks of manual aggregation	Real-time automated metrics
Verification	No formal workflow	Mandatory approval queue with evidence review
Data storage	Scattered spreadsheets and emails	Single PostgreSQL database with RLS
Audit trail	No systematic logging	Complete, immutable action log
Access control	Role confusion and ad hoc sharing	6-tier enforced RBAC with database-level security
Reporting visibility	Quarterly at best	Live WebSocket updates to all connected dashboards

Calculations	Error-prone manual computations	Database trigger automation (13 metrics)
Review process	Email-based, unstructured	Structured verification queue with evidence attachments
Geographic analysis	No spatial capability	Interactive Leaflet.js map with 5 GeoJSON layers
CBD reporting	Manual document preparation	One-click export in 3 formats (CSV, PDF, JSON)

1.7 Scope of the System

In Scope:

- User authentication and role-based access control (6 roles)
- 7 structured reporting tools (T01–T07) for all NBSAP sectors
- Mandatory data verification queue with approve/reject workflow
- Automated metric calculation (13 metrics) via database triggers
- National target progress tracking (22 targets, ~80 indicators)
- Interactive GIS map with district boundaries, protected areas, rivers, lakes
- GBIF species occurrence data integration
- RBIS (Rwanda Biodiversity Information System) integration
- Data export in CSV, PDF, and JSON formats
- Risk register and compliance tracking
- Immutable audit log
- In-app notification system with real-time WebSocket delivery
- User account lifecycle management (create, suspend, deactivate)

Out of Scope (Future Enhancement):

- Mobile native application (iOS/Android)
- Offline data collection with sync
- Multi-language support (Kinyarwanda, French)
- Automated CBD National Report generation
- SMS-based notifications
- Integration with Rwanda's e-Government portal

1.8 Major System Capabilities

#	Capability	Description
1	22 National Target Tracking	Full NBSAP 2025–2030 target set with progress bars, milestones, and linked indicators
2	~80 Indicator Framework	4-tier indicator hierarchy (Headline, Component, Binary, Complementary) following KM-GBF

3	7 Reporting Tools (T01–T07)	Structured data collection forms covering every NBSAP sector
4	Mandatory Verification Queue	Every submission reviewed before affecting national metrics
5	13 Automated Metrics	Database triggers compute forest/wetland restoration, finance, HWC, EIA compliance, etc.
6	Interactive GIS Map	30 districts, protected areas, rivers, lakes, GBIF species occurrences
7	6-Tier Access Control	Role-based permissions enforced at both client and database levels
8	Real-Time Updates	WebSocket push notifications to all connected dashboards
9	Multi-Format Export	CSV, PDF, JSON export for analysis and CBD reporting
10	RBIS Integration	8 biodiversity data streams linked to national indicators
11	Immutable Audit Log	Complete accountability trail for every system action
12	Weighted Progress System	Configurable tool weights determine each report's contribution to target progress

SECTION 2: SYSTEM OVERVIEW

2.1 Current System

The NBSAP Monitoring Dashboard is a production system deployed at <https://nbsap-dashboard-rw.vercel.app>. It is actively processing biodiversity data from institutions, ministries, and district offices across Rwanda.

Attribute	Value
System Name	Rwanda NBSAP Monitoring Dashboard
Version	1.0.0
Status	Production — Live and Operational
Architecture	Serverless BaaS (React + Supabase)
Database	PostgreSQL 15 (18 tables, 25+ RLS policies, 7 triggers, 22 migrations)
Frontend	React 18 + TypeScript + Vite (16 pages, 50+ components)
Codebase	~15,000+ lines of TypeScript
Deployment	Vercel CDN (auto-deploy on git push)
Backend	Supabase Cloud (Auth, Database, Storage, Realtime, Edge Functions)

2.2 System Objectives

1. **Centralise** all biodiversity monitoring data in a single authoritative database
2. **Standardise** data collection through structured reporting tools with validation
3. **Automate** metric computation, eliminating human calculation errors
4. **Verify** every data submission before it affects national metrics
5. **Visualise** geographic and temporal patterns through interactive dashboards
6. **Export** CBD-compliant indicator data for international reporting
7. **Audit** every system action for governance and accountability
8. **Integrate** with GBIF and RBIS for real-time biodiversity data enrichment

2.3 Core Modules

Module 1: Authentication & User Management

Manages user registration, login, session management, role assignment, account suspension/deactivation, and role change request workflows. Built on Supabase Auth with JWT tokens and automatic session refresh.

Module 2: Dashboard & Executive View

Provides a national-level overview displaying 13 automated system metrics, target progress summaries, recent approved reports, and geographic compliance data. Real-time updates via WebSocket.

Module 3: National Targets & Indicators

Displays all 22 NBSAP 2025–2030 targets organised by 4 KM-GBF Goals, with ~80 linked indicators across 4 tiers, implementation milestones, baseline data, and progress tracking.

Module 4: Reporting Toolkit (T01–T07)

Seven structured reporting tools enabling field-level data collection across all NBSAP sectors. Each tool has domain-specific form fields, evidence upload capability, and NBSAP target alignment.

Module 5: Verification Queue

Mandatory quality gate where authorised reviewers examine, approve, or reject pending submissions. Approved data triggers automated metric updates; rejected data returns to reporters with feedback.

Module 6: Interactive Map (GIS)

Leaflet.js-based map rendering 30 districts across 5 provinces with toggleable GeoJSON layers (protected areas, rivers, lakes) and GBIF species occurrence overlays.

Module 7: RBIS Dashboard

Real-time connection to the Rwanda Biodiversity Information System, displaying 8 data streams, connection status, metrics, and indicator-stream linkage tracking.

Module 8: Reports & Analytics

Aggregated data analysis with filtering, trend visualisation, and multi-format export (CSV, PDF, JSON) for internal analysis and CBD National Report preparation.

Module 9: Risk & Compliance

Biodiversity risk register with severity tracking and EIA compliance monitoring across districts and sectors.

Module 10: Audit & Governance

Immutable audit log recording every system action, notification system for workflow events, and user activity monitoring.

2.4 User Groups

User Group	System Role	Count	Primary Activities
REMA Administrators	dashboard_management	2–5	Full system administration, verification, user management, CBD reporting
Ministry Reporters	lead_government_ministry_reporting	10–20	Submit T01–T07 reports, verify submissions, access analytics
District Officers	local_reporting	30–60	Submit district-level reports, view own submissions
Policy Monitors	policy_monitoring	5–15	View strategic dashboards, risk register, analytics (read-only)
Development Partners	programme_alignment	10–30	View analytical dashboards, programme alignment (read-only)

Public Viewers	public_viewer	Unlimited	View dashboards, targets, indicators, map (read-only)
-----------------------	---------------	-----------	---

2.5 Reporting Requirements

Requirement	Implementation
Internal reporting	Dashboard metrics, target progress, district compliance — all available in real-time
Sector reporting	7 reporting tools (T01–T07) covering all NBSAP sectors
Financial reporting	T05 captures budget allocation, disbursement, and utilisation
Compliance reporting	T06 tracks EIA compliance; compliance records table tracks overall adherence
CBD reporting	Indicator data follows KM-GBF 3-tier structure; exportable in CSV, PDF, JSON
Audit reporting	Immutable audit log accessible to REMA Administrators

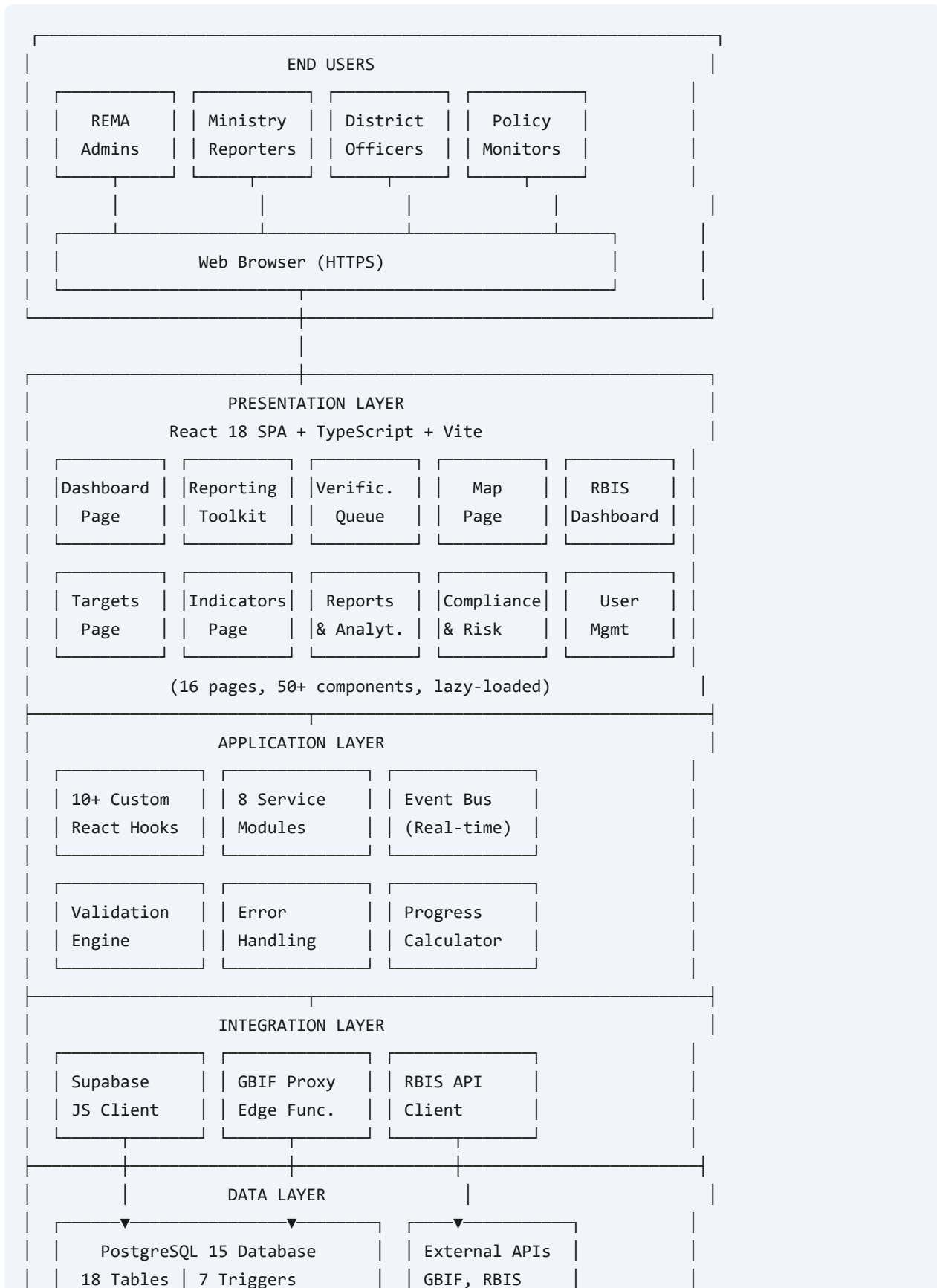
2.6 Monitoring Requirements

Requirement	Implementation
Target progress	Real-time progress bars with 3-colour status (green/amber/red) for all 22 targets
Indicator status	On-track / At-risk / Behind classification for ~80 indicators
District compliance	Compliance scores and forest cover percentages for all 30 districts
Species monitoring	GBIF occurrence data overlaid on interactive map
RBIS integration	8 data streams with connection status monitoring
System health	Data pipeline status, API connectivity, error monitoring

2.7 Dashboard Requirements

Requirement	Implementation
Executive metrics	13 automated metrics displayed with labels, values, and units
Target overview	22 targets with progress bars, organised by 4 KM-GBF Goals
Geographic view	Interactive map with 5 data layers and district-level detail
Recent activity	Latest approved reports displayed on dashboard
Real-time refresh	WebSocket push updates; no manual page refresh required

2.8 High-Level System Overview Diagram



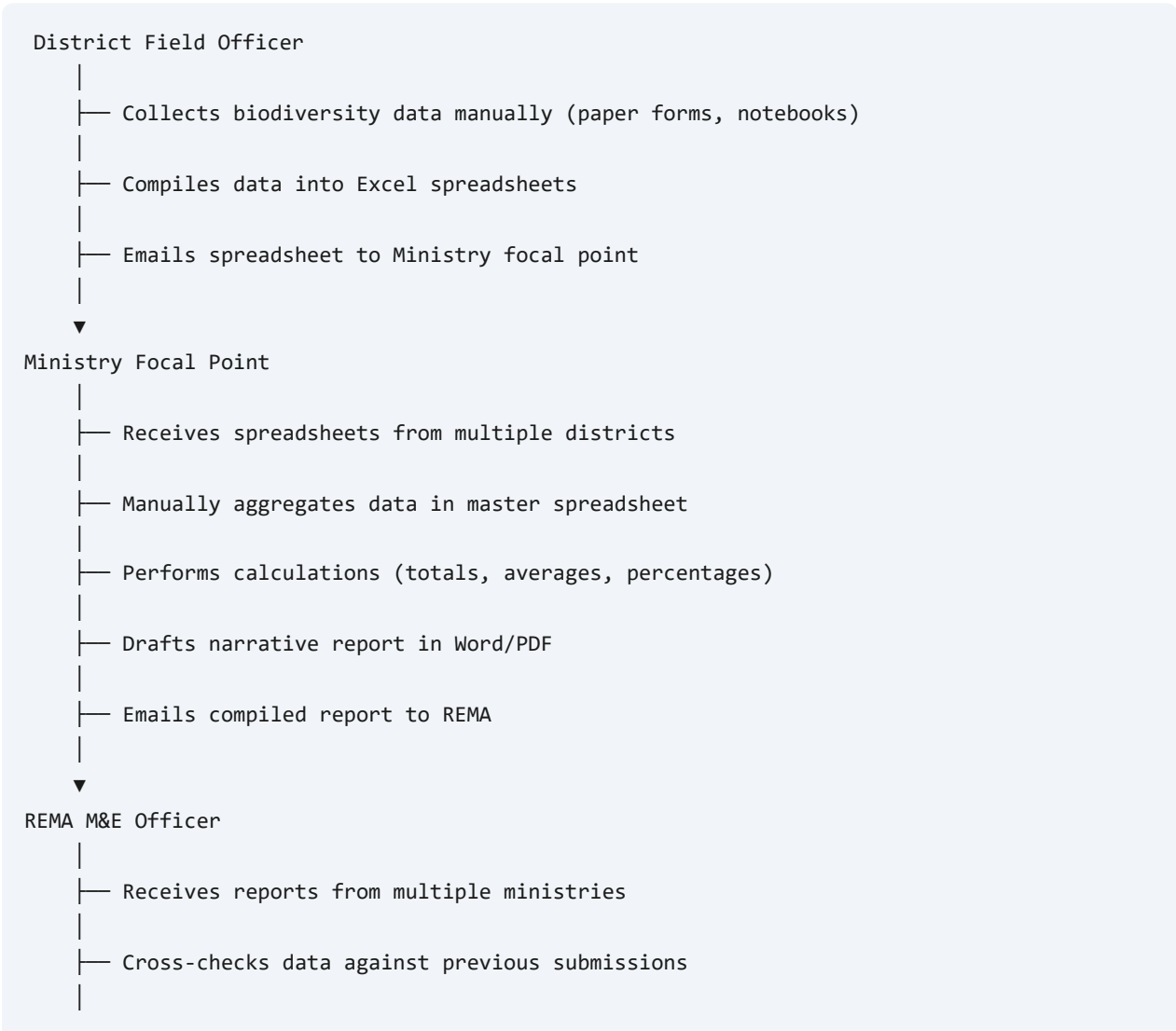
25+ RLS	10+ Functions	GeoJSON files
20+ Index	22 Migrations	
Supabase Auth (JWT/Sessions)	Supabase Storage (S3)	Supabase Realtime (WS)

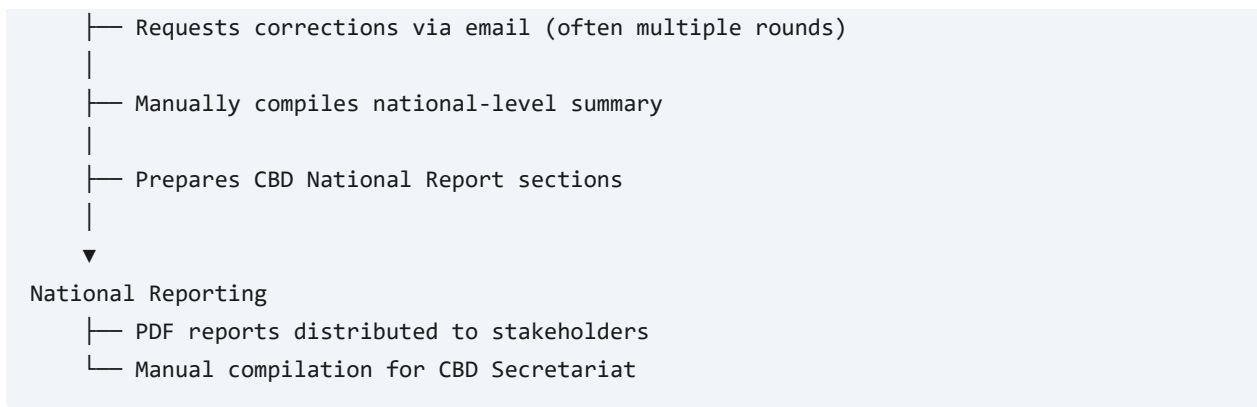
SECTION 3: BUSINESS PROCESS DOCUMENT

3.1 Current Process (AS-IS)

3.1.1 Pre-System Data Collection Process

Before the NBSAP Monitoring Dashboard, biodiversity data collection in Rwanda followed a manual, fragmented process:

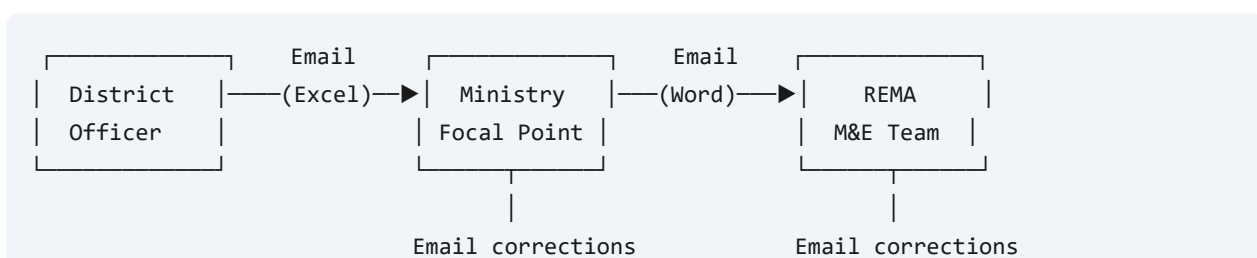


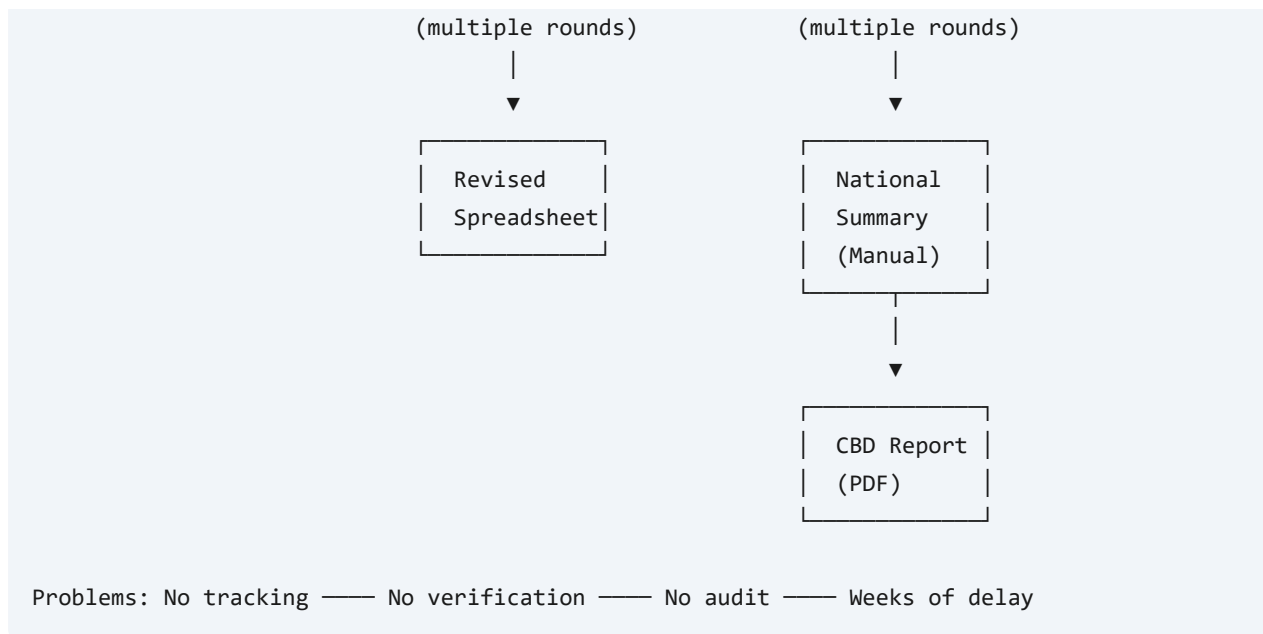


3.1.2 AS-IS Process Challenges and Bottlenecks

Challenge	Impact	Severity
No standard data format	Each ministry uses different spreadsheet structures; data aggregation is error-prone	High
Email-based review	Submissions lost, delayed, or duplicated; no tracking of review status	High
No verification workflow	Data published without formal quality assurance; errors propagate to national metrics	Critical
Manual calculations	Human computation errors in aggregating hectares, budgets, incident counts	High
No audit trail	Cannot determine who submitted, modified, or approved specific data points	Critical
Delayed visibility	National progress only visible quarterly (at best); no real-time monitoring	Medium
No geographic analysis	No spatial capability to visualise district-level patterns or biodiversity hotspots	Medium
Role confusion	No formal access control; anyone with email access can view or modify reports	High
Version control issues	Multiple versions of spreadsheets circulating; no single source of truth	High
CBD reporting burden	Weeks of manual compilation to prepare periodic National Reports	High

3.1.3 AS-IS Process Flow Diagram

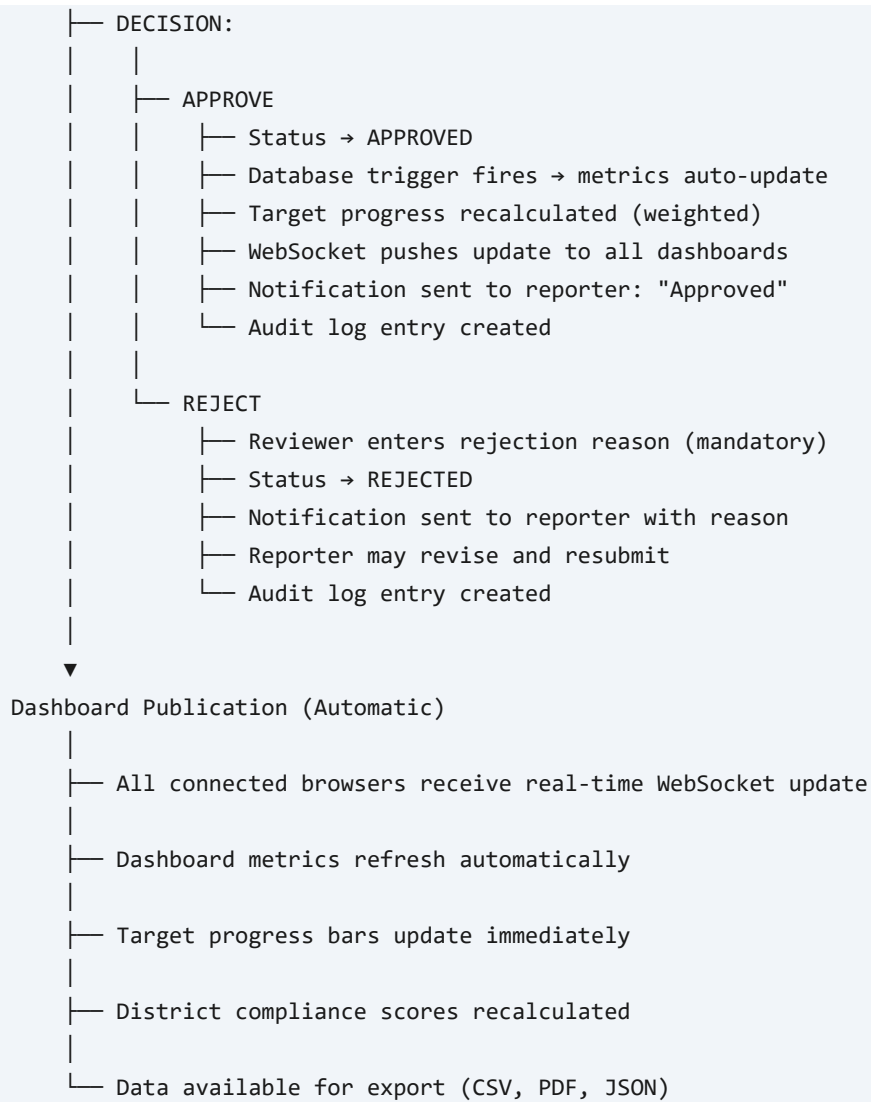




3.2 Proposed Process (TO-BE)

3.2.1 Automated Digital Process





3.2.2 Detailed Workflow Diagrams

Submission Workflow



```

|   |   | T05: budget_allocated, budget_disbursed, finance_source
|   |   | T06: company, eia_status, sector, restoration_commitments
|   |   | T07: community_name, district, participant_count, traditional_knowledge
|
| Reporter fills form fields
|
| Reporter selects reporting period (year validated: 2020-2030)
|
| Reporter selects NBSAP target alignment
|
| Reporter uploads evidence (optional attachments)
|
| Reporter clicks SUBMIT
|
| CLIENT-SIDE VALIDATION:
|   |   | Required fields present?
|   |   | Numeric values within valid ranges?
|   |   | Year within 2020-2030?
|   |   | PASS → proceed
|   |   | FAIL → display validation errors; block submission
|
| Supabase API call: INSERT into toolkit_reports
|   |   | RLS policy check (can_write() function)
|   |   | Database constraints check (NOT NULL, CHECK, FK)
|   |   | form_data stored as JSONB
|   |   | status = 'pending'
|
| Attachments uploaded to Supabase Storage
|
| Audit log entry: action_type='submit'
|
| Notification created for reviewers
|
END → Report visible in Verification Queue

```

Review Workflow

```

START
|
| Reviewer opens Verification Queue
|
| System checks: role has canApproveReports permission?
|   |   | NO → Page not accessible
|   |   | YES → Queue displayed
|
| Queue shows all pending submissions:
|   |   | Submitter name and role
|   |   | Tool type (T01-T07) with icon
|   |   | District and institution

```

```

|   |   | Submission date
|   |   | Status badge (Pending/Approved/Rejected)
|
| Reviewer selects a submission
|
| System displays:
|   |   | Full form_data fields and values
|   |   | Evidence attachments (downloadable)
|   |   | Reporter profile and organisation
|   |   | Historical submissions for same district/tool
|
| REVIEWER DECISION:
|   |
|   |   | APPROVE → UPDATE status='approved', reviewed_by, reviewed_at
|   |   |   | Triggers fire (see Approval Workflow below)
|   |   |
|   |   | REJECT → Reviewer enters rejection reason
|   |   |   | UPDATE status='rejected', review_note, reviewed_by, reviewed_at
|   |   |   | Notification to reporter with rejection reason
|   |   |   | Reporter can revise and resubmit (new submission)
|   |   |
|   |   | RESET TO PENDING → UPDATE status='pending' (undo previous decision)
|
END

```

Approval Workflow (Automated Post-Approval)

```

Report status changes to 'approved'
|
| Database trigger: on_report_approved fires
|
| Trigger extracts metric values from form_data JSONB:
|   |   | T02 → forest_hectares, wetland_hectares
|   |   | T03 → coverage_km2, mgmt_effectiveness, illegal_cases
|   |   | T04 → hwc_case_count
|   |   | T05 → budget_allocated, budget_disbursed
|   |   | T06 → eia_status (full/partial/non), restoration_ha
|   |   | T07 → participant_count, traditional_knowledge_count
|
| system_metrics table updated (INSERT or UPDATE)
|
| Weighted progress trigger fires:
|   |   | Lookup tool_weights for this tool and target
|   |   | Calculate: target_progress = SUM(tool_weight × tool_metric)
|   |   | UPDATE nbsap_targets.progress
|
| Supabase Realtime broadcasts change via WebSocket
|
| All connected browsers receive update

```

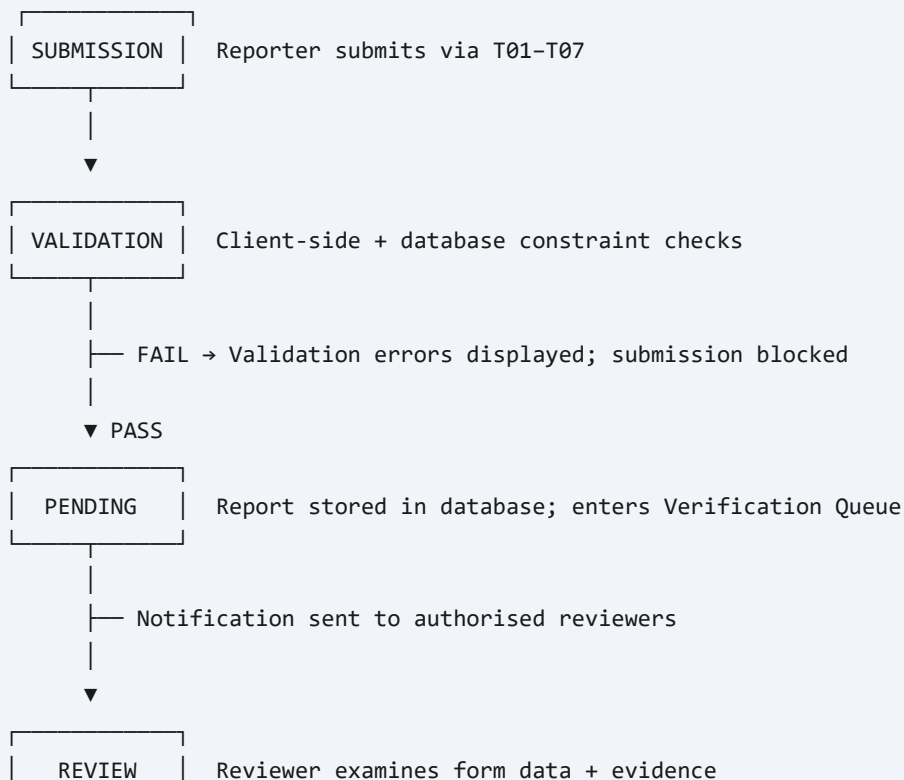

- └─ Dashboard metrics refresh
- └─ Target progress bars update
- └─ District compliance recalculated
- └─ Notification created: "Your report was approved"
- └─ Audit log entry: action_type='approve'

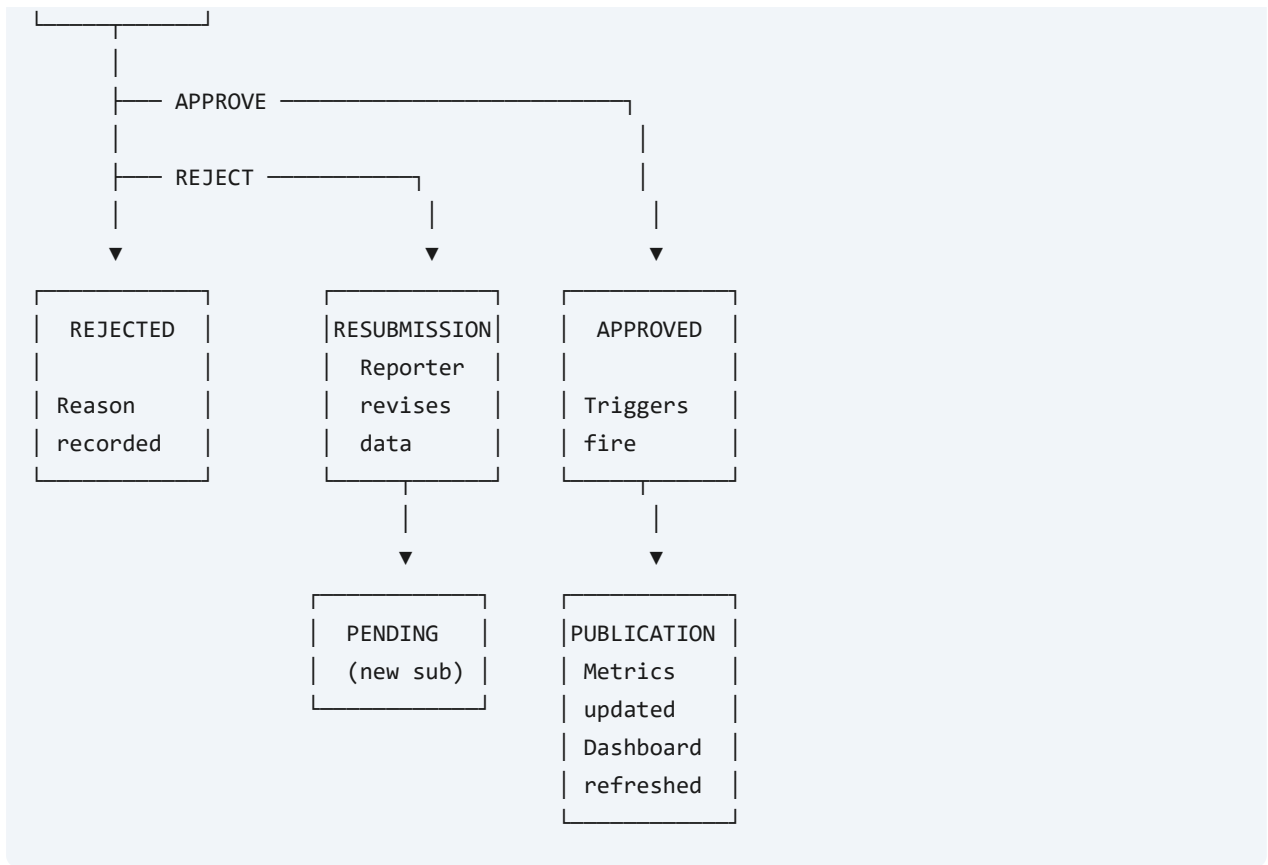
Escalation Workflow

- Submission remains PENDING for > 48 hours
- └─ (Future enhancement) System generates overdue alert
 - └─ REMA Administrator monitors queue aging
 - └─ Admin contacts designated reviewer
 - └─ If reviewer unavailable:
 - └─ Admin assigns alternate reviewer
 - └─ Admin may approve directly (dashboard_management role)

3.3 Approval Flow

3.3.1 Complete Approval Flow Diagram





SECTION 4: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

4.1 Functional Requirements

4.1.1 User Management

Req ID	Description	Inputs	Processing	Outputs	Priority
FR-UM-001	User Registration	Email, password	Supabase Auth creates account; trigger creates profiles row with default role (policy_monitoring)	User account created; confirmation	Must Have
FR-UM-002	User Authentication	Email, password	Supabase Auth validates credentials; issues JWT access + refresh tokens	Authenticated session; redirect to dashboard	Must Have
FR-UM-003	Password Reset	Email	Supabase Auth sends password reset email	Reset link sent to user's email	Must Have

FR-UM-004	Session Management	JWT tokens	Access token auto-refreshes via refresh token; no re-login during active use	Continuous authenticated session	Must Have
FR-UM-005	Role Assignment	User ID, new role	Admin updates profiles.role; RLS policies enforce new access level immediately	User access level changed	Must Have
FR-UM-006	Account Suspension	User ID, reason, optional end date	Admin sets suspended_at, suspension_reason, suspension_end_date	Account locked; user cannot login	Must Have
FR-UM-007	Auto-Reactivation	Suspension end date	System checks suspension_end_date on login; if expired, clears suspension	Account automatically unlocked	Should Have
FR-UM-008	Account Deactivation	User ID	Admin sets is_active=false	Account permanently locked	Must Have
FR-UM-009	Role Change Request	Current role, desired role, justification	User submits request; enters pending queue for admin review	Role change request created	Should Have
FR-UM-010	Role Change Approval	Request ID, decision, note	Admin approves/rejects; if approved, user role updated via trigger	User role updated or rejection sent	Should Have
FR-UM-011	User Profile Update	Full name, organisation, department, phone	User updates own profile fields (RLS: own record only)	Profile updated	Must Have
FR-UM-012	User Listing	Filters (role, status, search)	Admin queries all profiles with pagination	User list with metadata	Must Have

4.1.2 Dashboard Management

Req ID	Description	Inputs	Processing	Outputs	Priority
FR-DM-001	Dashboard Generation	None (auto)	System queries system_metrics, nbsap_targets, recent reports	Executive dashboard rendered	Must Have
FR-DM-002	13 Automated Metrics Display	None (auto)	Fetch from system_metrics table (trigger-computed)	Metric cards with values, labels, units	Must Have

FR-DM-003	Target Progress Tracking	None (auto)	Fetch all 22 targets with computed progress	Progress bars with 3-colour coding	Must Have
FR-DM-004	Red Flag Monitoring	Progress thresholds	Targets with progress < 40% flagged red; indicators marked "behind"	Visual red status badges	Must Have
FR-DM-005	Real-Time Update	WebSocket event	Supabase Realtime pushes changes; React state updates	Dashboard refreshes without page reload	Should Have
FR-DM-006	District Compliance Map	District data	Fetch districts with compliance scores; render on Leaflet map	Colour-coded district map	Must Have
FR-DM-007	Recent Reports Display	Approved reports	Fetch last 4 approved toolkit_reports with submitter info	Recent activity cards	Should Have

4.1.3 Reporting

Req ID	Description	Inputs	Processing	Outputs	Priority
FR-RP-001	T01 Submission	Institution, budget, compliance, policy progress	Validate fields; INSERT toolkit_reports with tool_id='T01'	Report stored with status='pending'	Must Have
FR-RP-002	T02 Submission	District, forest ha, wetland ha, land cover	Validate fields; INSERT with tool_id='T02'	Report stored with status='pending'	Must Have
FR-RP-003	T03 Submission	Area name, coverage, effectiveness, illegal cases	Validate fields; INSERT with tool_id='T03'	Report stored with status='pending'	Must Have
FR-RP-004	T04 Submission	Incident type, district, species, HWC count	Validate fields; INSERT with tool_id='T04'	Report stored with status='pending'	Must Have
FR-RP-005	T05 Submission	Budget allocated, disbursed, source	Validate fields; INSERT with tool_id='T05'	Report stored with status='pending'	Must Have
FR-RP-006	T06 Submission	Company, EIA status, sector, restoration	Validate fields; INSERT with tool_id='T06'	Report stored with status='pending'	Must Have
FR-RP-007	T07 Submission	Community, district, participants, TK	Validate fields; INSERT with tool_id='T07'	Report stored with status='pending'	Must Have

FR-RP-008	Evidence Upload	File (document/image)	Upload to Supabase Storage; link to report	Attachment stored and linked	Must Have
FR-RP-009	Report Approval	Report ID, reviewer decision	UPDATE status='approved'; trigger fires metric update	Metrics updated; notification sent	Must Have
FR-RP-010	Report Rejection	Report ID, rejection reason	UPDATE status='rejected'; notification with reason	Reporter notified with feedback	Must Have
FR-RP-011	Report Export (CSV)	Filters (tool, date, district)	Query filtered reports; generate CSV	CSV file downloaded	Must Have
FR-RP-012	Report Export (PDF)	Filters	Query filtered reports; generate formatted PDF	PDF file downloaded	Must Have
FR-RP-013	Report Export (JSON)	Filters	Query filtered reports; serialise to JSON	JSON file downloaded	Should Have
FR-RP-014	Report Deletion	Report ID	DELETE from toolkit_reports; if was approved, trigger reverses metrics	Report removed; metrics reversed	Must Have

4.1.4 Audit and Compliance

Req ID	Description	Inputs	Processing	Outputs	Priority
FR-AC-001	Audit Logging	Action type, action, detail, user ID	INSERT into audit_log (append-only, immutable)	Audit entry created	Must Have
FR-AC-002	Audit Log Viewing	Filters (user, action_type, date range)	Admin queries audit_log with filters	Filtered audit trail displayed	Must Have
FR-AC-003	Change Tracking	Before/after values	Audit entry includes detail field with change context	Change history queryable	Must Have
FR-AC-004	User Activity Monitoring	User ID	Query audit_log by user_id	User's activity history	Must Have
FR-AC-	Risk Register	Risk data (description,	CRUD operations on risks table	Risk register maintained	Should Have

005		category, level)			
FR-AC-006	Compliance Records	Compliance data	CRUD operations on compliance_records	Compliance tracking maintained	Should Have

4.2 Non-Functional Requirements

4.2.1 Performance

Req ID	Requirement	Target	Measurement Method
NFR-P-001	Initial page load time	< 3 seconds on 4G connection	Browser DevTools / Lighthouse
NFR-P-002	Dashboard metrics display	< 2 seconds	Supabase query latency
NFR-P-003	API response time (p50)	< 200ms	Supabase dashboard metrics
NFR-P-004	API response time (p95)	< 500ms	Supabase dashboard metrics
NFR-P-005	Map render time	< 2 seconds	Chrome Performance tab
NFR-P-006	Concurrent user capacity	500 users	Load testing
NFR-P-007	Report storage capacity	10,000+ reports	Database capacity planning
NFR-P-008	Search response time	< 500ms	Client-side measurement
NFR-P-009	Code splitting	Every page lazy-loaded	Bundle analysis
NFR-P-010	Asset caching	1-year cache headers on hashed assets	Response header inspection

4.2.2 Security

Req ID	Requirement	Implementation
NFR-S-001	HTTPS/TLS encryption	All traffic encrypted via TLS 1.2+ (Vercel + Supabase)
NFR-S-002	JWT authentication	Supabase Auth issues JWT access + refresh tokens
NFR-S-003	Password hashing	bcrypt via Supabase Auth

NFR-S-004	Row-Level Security	25+ PostgreSQL RLS policies on all 18 tables
NFR-S-005	Content Security Policy	CSP headers restrict script/style sources
NFR-S-006	Clickjacking protection	X-Frame-Options: DENY
NFR-S-007	MIME sniffing prevention	X-Content-Type-Options: nosniff
NFR-S-008	XSS protection	X-XSS-Protection: 1; mode=block
NFR-S-009	Immutable audit log	Append-only audit_log table; no UPDATE/DELETE allowed
NFR-S-010	Input validation	Client-side TypeScript + database constraints
NFR-S-011	File upload security	Supabase Storage with bucket-level access policies
NFR-S-012	Session management	JWT auto-refresh; stale session detection

4.2.3 Availability

Req ID	Requirement	Target
NFR-A-001	System uptime	99.5% annually
NFR-A-002	Database backup frequency	Daily (automated by Supabase)
NFR-A-003	Recovery Point Objective (RPO)	24 hours
NFR-A-004	Recovery Time Objective (RTO)	4 hours
NFR-A-005	Zero-downtime deployment	Vercel auto-deploy with instant rollback
NFR-A-006	Graceful degradation	System functional without GBIF/RBIS (external APIs)

4.2.4 Scalability

Req ID	Requirement	Implementation
NFR-SC-001	Horizontal scaling	Serverless architecture (Vercel + Supabase) auto-scales
NFR-SC-002	CDN distribution	Vercel serves static assets from 200+ global edge locations
NFR-SC-003	Database scaling	Supabase Pro tier: 8GB database, connection pooling (PgBouncer)
NFR-SC-004	Additional institutions	New organisations added via user management; no code changes
NFR-SC-005	Additional indicators	Indicators table supports unlimited rows; UI paginated
NFR-SC-006	Storage scaling	Supabase Pro tier: 100GB file storage

4.2.5 Usability

Req ID	Requirement	Implementation
--------	-------------	----------------

NFR-U-001	WCAG 2.1 AA compliance	Colour contrast 4.5:1 minimum; ARIA labels on all interactive elements
NFR-U-002	Keyboard navigation	Full Tab/Enter/Space/Escape keyboard support
NFR-U-003	Screen reader support	Semantic HTML; aria-live regions for real-time updates
NFR-U-004	Responsive design	Adaptive layout: desktop (>1024px), tablet (768–1024px), mobile (<768px)
NFR-U-005	Consistent visual language	3-colour progress system (green/amber/red) used consistently
NFR-U-006	Loading states	Skeleton screens during data fetch; loading spinners for actions
NFR-U-007	Error messages	User-friendly error messages with actionable guidance
NFR-U-008	Browser compatibility	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+

SECTION 5: USER ROLES AND PERMISSIONS

5.1 Role Definitions

5.1.1 Administrator — `dashboard_management`

Attribute	Value
Label	REMA Administrator
Typical Users	REMA national team, system administrators
Description	Full system access: verification queue, user management, audit log, all exports
User Count	2–5
Permissions	ALL (canSubmitReports, canApproveReports, canViewVerifQueue, canViewAuditLog, canManageUsers, canExportRaw, canViewRiskRegister, canViewCompliance, canViewAnalytics)

5.1.2 REMA M&E Officer / Lead Ministry Reporter —

`lead_government_ministry_reporting`

Attribute	Value
Label	Lead Government Ministry Reporter
Typical Users	Ministry focal points, sector ministry officers
User Count	10–20
Permissions	canSubmitReports, canApproveReports, canViewVerifQueue, canExportRaw, canViewRiskRegister, canViewCompliance, canViewAnalytics

5.1.3 Sector Reviewer / District Officer — `local_reporting`

Attribute	Value
Label	Local Reporter
Typical Users	District environmental officers (30 districts)
User Count	30–60
Permissions	canSubmitReports, canViewCompliance

5.1.4 Institution Reporter / Policy Monitor — `policy_monitoring`

Attribute	Value
Label	Policy Monitor
Typical Users	National policymakers, Cabinet-level advisors, MoE oversight
User Count	5–15
Permissions	canViewRiskRegister, canViewCompliance, canViewAnalytics (READ-ONLY)

5.1.5 Viewer / Development Partner — `programme_alignment`

Attribute	Value
Label	Development Partner
Typical Users	NGOs, research institutions, donors
User Count	10–30
Permissions	canViewRiskRegister, canViewCompliance, canViewAnalytics (READ-ONLY)

5.1.6 Auditor / Public Viewer — `public_viewer`

Attribute	Value
-----------	-------

Label	Public Viewer
Typical Users	General public, media, academic researchers
User Count	Unlimited
Permissions	View dashboard, targets, indicators, map ONLY (no other permissions)

5.2 Detailed Permission Matrix

Permission	REMA Admin	Ministry Reporter	District Officer	Policy Monitor	Dev Partner	Public Viewer
canSubmitReports	YES	YES	YES	NO	NO	NO
canApproveReports	YES	YES	NO	NO	NO	NO
canViewVerifQueue	YES	YES	NO	NO	NO	NO
canViewAuditLog	YES	NO	NO	NO	NO	NO
canManageUsers	YES	NO	NO	NO	NO	NO
canExportRaw	YES	YES	NO	NO	NO	NO
canViewRiskRegister	YES	YES	NO	YES	YES	NO
canViewCompliance	YES	YES	YES	YES	YES	NO
canViewAnalytics	YES	YES	NO	YES	YES	NO
View Dashboard	YES	YES	YES	YES	YES	YES
View Targets	YES	YES	YES	YES	YES	YES
View Indicators	YES	YES	YES	YES	YES	YES
View Map	YES	YES	YES	YES	YES	YES
View RBIS	YES	YES	YES	YES	YES	YES
User Management	YES	NO	NO	NO	NO	NO
Role Requests	YES	NO	NO	NO	NO	NO
Data Pipeline	YES	NO	NO	NO	NO	NO
Stakeholders	YES	NO	NO	NO	NO	NO
Settings (own)	YES	YES	YES	YES	YES	YES

SECTION 6: BUSINESS RULES

6.1 Data Integrity Rules

Rule ID	Business Rule	Enforcement
BR-001	Every indicator must belong to exactly one NBSAP target	<code>indicators.nbsap_target_id</code> FK constraint to <code>nbsap_targets.id</code>
BR-002	Progress values cannot exceed 100% or go below 0%	Database CHECK constraint: <code>progress BETWEEN 0 AND 100</code>
BR-003	Compliance scores must be between 0 and 100	Database CHECK constraint: <code>compliance BETWEEN 0 AND 100</code>
BR-004	Forest cover percentage must be between 0 and 100	Database CHECK constraint: <code>forest_cover BETWEEN 0 AND 100</code>
BR-005	Tool weights for any target must sum to 1.0	Application-level validation in <code>validateToolWeights()</code>
BR-006	Individual tool weights must be between 0.0 and 1.0	Application-level validation in <code>validateToolWeightValue()</code>
BR-007	Reporting year must fall within 2020–2030	Validated by <code>validateYear()</code> utility function
BR-008	Reporting dates must fall within 2020-01-01 to 2030-12-31	Validated by <code>validateDate()</code> utility function

6.2 Workflow Rules

Rule ID	Business Rule	Enforcement
BR-009	All submitted reports default to 'pending' status	Database DEFAULT on <code>toolkit_reports.status</code>
BR-010	No submitted data affects national metrics until approved	Metric triggers only fire on <code>status= 'approved'</code>
BR-011	Only users with <code>canApproveReports</code> permission can approve or reject reports	Client-side route guard + RLS policy
BR-012	Rejection requires a written reason	Client-side validation (note field mandatory for reject action)
BR-013	Only users with <code>canSubmitReports</code> permission can submit reports	Client-side route guard + <code>can_write()</code> RLS function
BR-014	Only REMA Administrators can manage user accounts	Client-side route guard + <code>is_admin()</code> RLS function

BR-015	A reviewer should not approve their own submission	Application-level check (separation of duties)
BR-016	Role change requests require administrator approval	Workflow enforced via <code>role_change_requests</code> table
BR-017	Deleting an approved report automatically reverses its metric impact	Database trigger: <code>on_report_deleted</code>

6.3 Access Control Rules

Rule ID	Business Rule	Enforcement
BR-018	Users can only view their own profile (unless admin)	RLS policy: <code>auth.uid() = id</code> OR <code>is_admin()</code>
BR-019	Users can only read their own notifications	RLS policy: <code>auth.uid() = user_id</code>
BR-020	Only administrators can view the audit log	RLS policy: <code>is_admin()</code>
BR-021	Suspended accounts cannot log in	<code>getAccountStatus()</code> check on <code>auth</code> ; <code>canLogin: false</code>
BR-022	Deactivated accounts cannot log in	ProtectedRoute checks <code>user.is_active</code>
BR-023	Suspended accounts auto-reactivate when end date passes	<code>getAccountStatus()</code> returns <code>suspended_expired</code> with <code>canLogin: true</code>
BR-024	Public Viewers have no write access to any table	RLS: <code>can_write()</code> excludes <code>public_viewer</code> role

6.4 Calculation Rules

Rule ID	Business Rule	Enforcement
BR-025	Indicator status is derived from progress: >=70% = on-track, 40–69% = at-risk, <40% = behind	<code>updateIndicatorProgress()</code> service function
BR-026	Target progress is calculated as the weighted sum of tool contributions	<code>calculate_weighted_progress()</code> database function
BR-027	System metrics are summed/averaged from approved reports only	Database trigger queries <code>toolkit_reports</code> WHERE <code>status='approved'</code>

BR-028	EIA compliance rate = (full compliance count / total EIA assessments) × 100	<code>systemMetricsService.ts</code> calculation
BR-029	Budget disbursement rate = (total disbursed / total allocated) × 100	Database trigger calculation
BR-030	All metric computations are trigger-driven; manual computation is not permitted	Architecture design principle

SECTION 7: DATA REQUIREMENTS

7.1 Entity Definitions

7.1.1 Targets

Field	Type	Constraints	Description
id	INTEGER	PK, 1–22	Target number
goal	ENUM (A,B,C,D)	NOT NULL	KM-GBF Goal alignment
title	TEXT	NOT NULL	Official target title
description	TEXT		Full target statement
baseline	TEXT		Baseline data description
headline_indicator	TEXT		Primary headline indicator
timeline_milestones	TEXT		Phase-by-phase milestones 2025–2030
responsible_stakeholders	TEXT[]		Array of responsible institutions
progress	INTEGER	CHECK 0–100	Auto-calculated progress percentage

7.1.2 Indicators

Field	Type	Constraints	Description
id	SERIAL	PK	Unique identifier
name	TEXT	NOT NULL	Indicator name
definition	TEXT		Detailed definition
tier	ENUM	NOT NULL	headline / component / complementary / binary
nbsap_target_id	INTEGER	FK → nbsap_targets	Parent target
baseline	TEXT		Baseline value

current_value	TEXT		Current measured value
midterm	TEXT		Midterm target
final_target	TEXT		2030 target value
progress	INTEGER	CHECK 0–100	Progress percentage
status	ENUM		on-track / at-risk / behind
km_gbf	TEXT		KM-GBF indicator reference
periodicity	TEXT		Measurement frequency
data_source	TEXT		Data source description
responsible	TEXT[]		Responsible institutions

7.1.3 Progress Records (toolkit_reports)

Field	Type	Constraints	Description
id	UUID	PK	Unique report identifier
tool_id	ENUM	NOT NULL	T01–T07
tool_name	TEXT	NOT NULL	Human-readable tool name
submitted_by	UUID	FK → profiles	Reporter user ID
status	ENUM	DEFAULT 'pending'	pending / approved / rejected
reviewed_by	UUID	FK → profiles	Reviewer user ID
form_data	JSONB	NOT NULL	Dynamic form fields
attachments	JSONB		Attachment metadata
district	TEXT		District (denormalised)
institution	TEXT		Institution (denormalised)
nbsap_target_id	INTEGER	FK → nbsap_targets	Linked target

7.1.4 Users (profiles)

Field	Type	Constraints	Description
id	UUID	PK, FK → auth.users	User unique identifier
email	TEXT	NOT NULL	User email address
full_name	TEXT		Full name
role	ENUM	NOT NULL	One of 6 roles

organization	TEXT		Organisation name
is_active	BOOLEAN	DEFAULT TRUE	Account active status
suspended_at	TIMESTAMPTZ		Suspension timestamp
suspension_reason	TEXT		Reason for suspension
suspension_end_date	TIMESTAMPTZ		Auto-reactivation date

7.1.5 Evidence Documents (report_attachments)

Field	Type	Constraints	Description
id	UUID	PK	Unique attachment ID
report_id	UUID	FK → toolkit_reports (CASCADE)	Parent report
file_name	TEXT	NOT NULL	Original filename
file_ext	TEXT		File extension
file_size	BIGINT		Size in bytes
storage_path	TEXT		Supabase Storage path
uploaded_by	UUID	FK → profiles	Uploader

7.1.6 Funding Sources and Budgets (via T05 form_data)

Field	Type	Stored In	Description
budget_allocated	NUMERIC	toolkit_reports.form_data	Budget allocated (RWF)
budget_disbursed	NUMERIC	toolkit_reports.form_data	Budget disbursed (RWF)
budget_utilized	NUMERIC	toolkit_reports.form_data	Budget utilised (RWF)
finance_source	TEXT	toolkit_reports.form_data	Funding source description

7.1.7 Audit Logs

Field	Type	Constraints	Description
id	UUID	PK	Log entry ID
user_id	UUID	FK → profiles	Actor
action_type	TEXT	NOT NULL	submit / approve / reject / export / login / delete
action	TEXT	NOT NULL	Human-readable description
detail	TEXT		Additional context

role	TEXT		User's role at time of action
ip_address	TEXT		Client IP
created_at	TIMESTAMPTZ		Timestamp

7.1.8 Geographic Entities

Districts (30 rows): id, name, province_id, status, compliance, forest_cover, latitude, longitude

Provinces (5 rows): id, name — Kigali, South, North, West, East

Lakes: id, name, province, area_km2, latitude, longitude

7.1.9 Risk Register

Field	Type	Description
id	TEXT	Risk identifier
description	TEXT	Risk description
category	TEXT	Risk category
likelihood	TEXT	Likelihood assessment
impact	TEXT	Impact assessment
level	ENUM	High / Medium / Low
mitigation	TEXT	Mitigation strategy
owner	TEXT	Risk owner
is_live	BOOLEAN	Auto-generated from submissions

SECTION 8: USE CASE SPECIFICATION

UC-001: User Login

Attribute	Description
Use Case ID	UC-001
Title	User Login
Actors	All users
Preconditions	User has registered account; account is active

Main Flow	1. User navigates to login page 2. User enters email and password 3. System validates credentials via Supabase Auth 4. System issues JWT access token + refresh token 5. System fetches user profile from <code>profiles</code> table 6. System checks account status (active/suspended/deactivated) 7. System redirects to dashboard
Alternative Flow	3a. Invalid credentials → display error message 6a. Account suspended → display suspension message with reason 6b. Account deactivated → display deactivation message
Exceptions	Network failure → display connection error; Supabase unavailable → display service error
Postconditions	User has authenticated session; audit log entry created (action_type='login')

UC-002: Submit Indicator Data

Attribute	Description
Use Case ID	UC-002
Title	Submit Indicator Data via Reporting Tool
Actors	REMA Administrator, Ministry Reporter, District Officer
Preconditions	User authenticated; user has <code>canSubmitReports</code> permission
Main Flow	1. User navigates to Reporting Toolkit 2. User selects tool (T01–T07) 3. System renders tool-specific form 4. User fills required fields 5. User selects NBSAP target and district 6. User selects reporting period (year) 7. User clicks Submit 8. System validates all inputs 9. System creates <code>toolkit_report</code> with status='pending' 10. System creates audit log entry 11. System sends notification to reviewers
Alternative Flow	8a. Validation fails → display field-level errors; block submission
Exceptions	Storage failure on attachment upload → display error; retry
Postconditions	Report stored in database with status='pending'; visible in Verification Queue

UC-003: Upload Evidence

Attribute	Description
Use Case ID	UC-003
Title	Upload Evidence Document
Actors	REMA Administrator, Ministry Reporter, District Officer
Preconditions	User is in the process of submitting a report (UC-002)
Main Flow	1. User clicks "Attach Evidence" 2. File picker opens 3. User selects file (document, image) 4. System validates file type and size 5. System uploads file to Supabase Storage 6.

	System links attachment to report
Alternative Flow	4a. Invalid file type → display error; reject file 4b. File too large → display size limit error
Exceptions	Upload network failure → retry with exponential backoff
Postconditions	File stored in Supabase Storage; metadata recorded in report_attachments

UC-004: Review Submission

Attribute	Description
Use Case ID	UC-004
Title	Review Pending Submission
Actors	REMA Administrator, Ministry Reporter (reviewer)
Preconditions	User authenticated; user has <code>canApproveReports</code> permission; pending submissions exist
Main Flow	1. Reviewer opens Verification Queue 2. System displays all pending submissions 3. Reviewer selects a submission 4. System displays full form data, evidence, submitter info 5. Reviewer examines data for completeness and plausibility 6. Reviewer downloads and reviews evidence attachments
Alternative Flow	No pending submissions → display "Queue empty" message
Exceptions	None
Postconditions	Reviewer has examined submission; ready to approve or reject (UC-005)

UC-005: Approve Submission

Attribute	Description
Use Case ID	UC-005
Title	Approve Submission
Actors	REMA Administrator, Ministry Reporter (reviewer)
Preconditions	Reviewer has examined submission (UC-004); submission is in 'pending' status
Main Flow	1. Reviewer clicks "Approve" 2. System updates status to 'approved' 3. System records reviewer ID and timestamp 4. Database trigger fires: metrics extracted from form_data 5. system_metrics table updated 6. Weighted progress trigger fires: target progress recalculated 7. WebSocket pushes update to all dashboards 8. Notification sent to reporter: "Your report was approved" 9. Audit log entry created

Alternative Flow	None
Exceptions	Database trigger failure → log error; alert admin
Postconditions	Report approved; national metrics updated; all dashboards reflect new data

UC-006: Generate Dashboard

Attribute	Description
Use Case ID	UC-006
Title	Generate Executive Dashboard
Actors	All authenticated users
Preconditions	User authenticated
Main Flow	1. User navigates to Dashboard 2. System queries system_metrics (13 automated metrics) 3. System queries nbsap_targets (22 targets with progress) 4. System queries recent approved reports (last 4) 5. System renders metric cards, progress bars, recent activity 6. WebSocket connection established for real-time updates
Alternative Flow	No data available → display zero values with "No data yet" labels
Exceptions	Database query failure → display error boundary with retry button
Postconditions	Dashboard displayed with current national biodiversity status

UC-007: View Red Flags

Attribute	Description
Use Case ID	UC-007
Title	View Red Flag Indicators and Targets
Actors	All authenticated users
Preconditions	User authenticated; dashboard or targets page loaded
Main Flow	1. System identifies targets with progress < 40% (red status) 2. System identifies indicators with status = 'behind' 3. Red badges and progress bars displayed prominently 4. User can drill down into specific targets/indicators
Alternative Flow	No red flags → all items show green/amber status
Exceptions	None
Postconditions	User aware of areas requiring urgent intervention

UC-008: Export Report

Attribute	Description
Use Case ID	UC-008
Title	Export Data Report
Actors	REMA Administrator, Ministry Reporter
Preconditions	User has <code>canExportRaw</code> permission
Main Flow	1. User navigates to Reports page 2. User selects filters (tool, date range, district, target) 3. User selects export format (CSV / PDF / JSON) 4. System queries filtered data 5. System generates export file 6. File downloaded to user's device 7. Audit log entry created (action_type='export')
Alternative Flow	No data matches filters → display "No results" message
Exceptions	Large dataset → pagination or streaming download
Postconditions	Export file on user's device; audit trail records the export

UC-009: Manage Users

Attribute	Description
Use Case ID	UC-009
Title	Manage User Accounts
Actors	REMA Administrator
Preconditions	User has <code>canManageUsers</code> permission
Main Flow	1. Admin opens User Management page 2. System displays all user profiles 3. Admin can: change role, suspend, deactivate, view activity 4. Changes take effect immediately (RLS policies enforce) 5. Audit log entries created for each action
Alternative Flow	Admin suspends with end date → auto-reactivation scheduled
Exceptions	Admin cannot deactivate own account → system prevents self-lockout
Postconditions	User account updated; audit trail records changes

SECTION 9: SYSTEM BEHAVIOUR

9.1 Data Submission Behaviour

User fills form → Client validates → API call to Supabase

- |
- | — RLS policy check: `can_write()` → TRUE?
 - | — NO → 403 Forbidden
 - | — YES → Continue
- |
- | — Database constraints: NOT NULL, CHECK, FK
 - | — FAIL → 400 Bad Request with constraint error
 - | — PASS → INSERT succeeds
- |
- | — Row created: `status='pending'`
- |
- | — Trigger: audit log entry created
- |
- | — Trigger: notification created for reviewers
- |
- | — Response: 201 Created with report ID

9.2 Approval Behaviour

Reviewer approves report → UPDATE `status='approved'`

- |
- | — Trigger: `on_report_approved`
 - | — Extract metric values from `form_data` JSONB
 - | — UPDATE `system_metrics` (increment values)
 - | — UPDATE `nbsap_targets.progress` (weighted calculation)
- |
- | — Supabase Realtime: broadcast change event
- |
- | — All connected browsers: React state updates
 - | — Dashboard metrics refresh
 - | — Target progress bars update
 - | — Event bus emits `'target-progress-updated'`
- |
- | — Notification: "Your report was approved"

9.3 Dashboard Calculation Behaviour

System loads dashboard:

- |
- | — Fetch `system_metrics` → 13 metric cards
 - | (forest_ha, wetland_ha, pa_coverage, mgmt_effectiveness,
 - | illegal_cases, hwc_incidents, budget_allocated, budget_disbursed,

```

    disbursement_rate, eia_compliance, restoration_ha,
    community_participants, tk_entries)

    └─ Fetch nbsap_targets → 22 targets with progress
        └─ Progress >= 70% → GREEN (on track)
        └─ Progress 40-69% → AMBER (at risk)
        └─ Progress < 40% → RED (behind)

    └─ Fetch recent approved reports → last 4

    └─ Establish WebSocket → auto-refresh on changes

```

9.4 Red Flag Generation Behaviour

Red flags are generated automatically based on data thresholds:

Trigger	Condition	Display
Target behind schedule	<code>nbsap_targets.progress < 40</code>	Red progress bar + "Behind" badge
Indicator off-track	<code>indicators.status = 'behind'</code>	Red status badge
District non-compliant	<code>districts.compliance < 50</code>	Red compliance indicator on map
District not reporting	<code>districts.status = 'missing'</code>	Grey "Missing" status on map
High-risk item active	<code>risks.level = 'High' AND risks.is_live = TRUE</code>	Red risk card in risk register

9.5 Report Generation Behaviour

```

User requests export:
    └─ Apply filters (tool, date, district, target)
    └─ Query toolkit_reports with filters
    └─ Format selection:
        └─ CSV → Generate comma-separated file with headers
        └─ PDF → Render formatted report with tables and charts
        └─ JSON → Serialize to structured JSON with metadata

    └─ Browser download triggered

    └─ Audit log: action_type='export'

```

9.6 Notification Behaviour

Event	Recipient	Notification Content
Report submitted	Authorised reviewers	"New [tool_name] submission from [reporter] for [district]"
Report approved	Reporter	"Your [tool_name] report has been approved"
Report rejected	Reporter	"Your [tool_name] report was rejected. Reason: [review_note]"
Role change approved	Requester	"Your role has been changed to [new_role]"
Role change rejected	Requester	"Your role change request was rejected. Reason: [note]"
Account suspended	User	"Your account has been suspended. Reason: [reason]"

Notifications are delivered via:

- 1. **In-app** — notifications table with is_read tracking
- 2. **Real-time** — WebSocket push via Supabase Realtime (no page refresh needed)

SECTION 10: VALIDATION RULES

10.1 Required Fields

Entity	Required Fields
User Registration	email, password
Profile Update	full_name
T01 Report	institution, compliance_score
T02 Report	district, forest_hectares
T03 Report	area_name, coverage_km2
T04 Report	incident_type, district, species
T05 Report	budget_allocated, finance_source
T06 Report	company, eia_status, sector
T07 Report	community_name, district, participant_count
All Reports	tool_id, tool_name, reporting period
Report Rejection	review_note (rejection reason)

Role Change Request	requested_role, justification
---------------------	-------------------------------

10.2 Date Validation

Rule	Implementation
Reporting year must be 2020–2030	<code>validateYear()</code> : checks <code>REPORTING_PERIOD.START_YEAR</code> to <code>END_YEAR</code>
Dates must be valid Date objects	<code>validateDate()</code> : parses string to Date; checks <code>isNaN()</code>
Dates must fall within reporting period	<code>validateDate()</code> : checks <code>MIN_DATE</code> to <code>MAX_DATE</code> (2020-01-01 to 2030-12-31)
Year must be an integer	<code>validateYear()</code> : checks <code>Number.isInteger()</code>

10.3 Numeric Range Validation

Field	Valid Range	Error Message
Progress value	0–100	"Progress value [X] is outside the 0-100 range"
Tool weight	0.0–1.0	"Tool weight [X] is outside the 0-1 range"
Tool weights sum	Must equal 1.0	"Tool weights sum to [X], expected 1.0"
Compliance score	0–100	Database CHECK constraint
Forest cover	0–100	Database CHECK constraint
Hectares	≥ 0	Application validation
Budget values	≥ 0	Application validation
Participant count	≥ 0	Application validation

10.4 Duplicate Detection

Scenario	Detection Method
Duplicate user email	Supabase Auth enforces unique email
Duplicate district name	Database UNIQUE constraint on <code>districts.name</code>
Duplicate province name	Database UNIQUE constraint on <code>provinces.name</code>
Duplicate indicator for same target	Application-level check during seed/import

10.5 Evidence Validation

Rule	Implementation
File type allowed	Document formats (PDF, DOC, DOCX, XLS, XLSX) and images (JPG, PNG)
File size limit	Application-level check before upload
Storage path generated	System generates unique path in Supabase Storage

10.6 Progress Clamping

When computing progress values, the system applies automatic clamping:

```
export function clampProgress(value: number): number {
  return Math.min(100, Math.max(0, value));
}
```

This ensures that even if a calculated value exceeds boundaries (e.g., 105% due to over-reporting), the stored value remains within the valid 0–100 range.

SECTION 11: ERROR HANDLING

11.1 Error Classification

The system uses a structured error code taxonomy defined in `errorHandling.ts`:

Category	Code Range	Examples
Authentication	AUTH_001 – AUTH_006	Unauthorized, invalid credentials, session expired, account deactivated, suspended, insufficient permissions
Validation	VAL_001 – VAL_006	Invalid input, required field missing, invalid format, out of range, invalid date, invalid year
Network	NET_001 – NET_003	Connection error, timeout, server error
API	API_001 – API_003	Request failed, invalid response, rate limit exceeded
Database	DB_001 – DB_005	Query failed, record not found, duplicate entry, constraint violation, connection error
File	FILE_001 – FILE_005	Not found, upload failed, invalid type, size exceeded, download failed

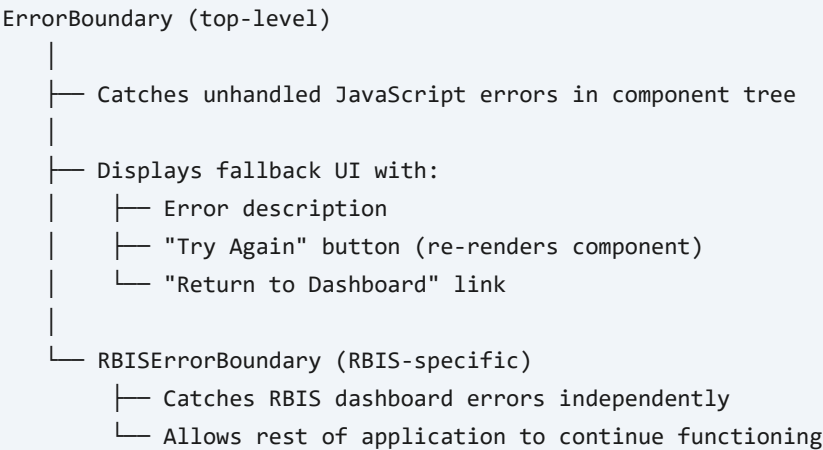
Business Logic	BIZ_001 – BIZ_003	Invalid operation, workflow error, state conflict
Unknown	ERR_000	Unclassified errors

11.2 Error Handling by Scenario

Scenario	System Response	User Message
Missing required data	Validation error; form submission blocked	"Please fill in all required fields" (field-level error highlights)
Invalid numeric data	Validation error; field rejected	"Value must be a number between [min] and [max]"
Authentication failure	Redirect to login; session cleared	"Invalid email or password. Please try again."
Authorization failure	403 response; access denied page	"Your role ([role]) does not have permission to view this page."
Network failure	Retry with exponential backoff; timeout after 30s	"Connection error. Please check your internet and try again."
Database query failure	Error logged to console; error boundary displayed	"Failed to load data. Please try again." (with retry button)
File upload failure	Upload retried; error displayed if persistent	"File upload failed. Please try again."
File type invalid	Upload rejected client-side	"Invalid file type. Allowed: PDF, DOC, DOCX, XLS, XLSX, JPG, PNG"
File size exceeded	Upload rejected client-side	"File is too large. Maximum size: [limit]"
GBIF API timeout	Graceful degradation; cached data shown	Map functions without species data; metrics show last-known values
RBIS API failure	Connection status indicator turns red	"RBIS connection failed. Dashboard functions with local data."
Supabase service down	Full system offline	"Service temporarily unavailable. Please try again later."
Stale chunk error	Auto-reload triggered	Page automatically refreshes to load new deployment

11.3 Error Boundary Components

The system uses React Error Boundaries to catch rendering errors:



SECTION 12: SYSTEM ARCHITECTURE

12.1 Technology Stack

Frontend

Technology	Version	Purpose
React	18.2	Component-based UI framework
TypeScript	5.0	Static type checking and code safety
Vite	4.3	Build tool and development server
React Router	v6	Client-side routing with protected routes
Leaflet.js	Latest	Interactive map rendering
Recharts	Latest	Chart and data visualisation
Lucide React	Latest	Icon library
React Hot Toast	Latest	User notification toasts

Backend

Technology	Version	Purpose
PostgreSQL	15	Relational database with JSONB, RLS, triggers
Supabase Auth	Latest	JWT-based authentication and session management
Supabase Storage	Latest	S3-compatible file storage for evidence attachments
Supabase Realtime	Latest	WebSocket-based real-time data push

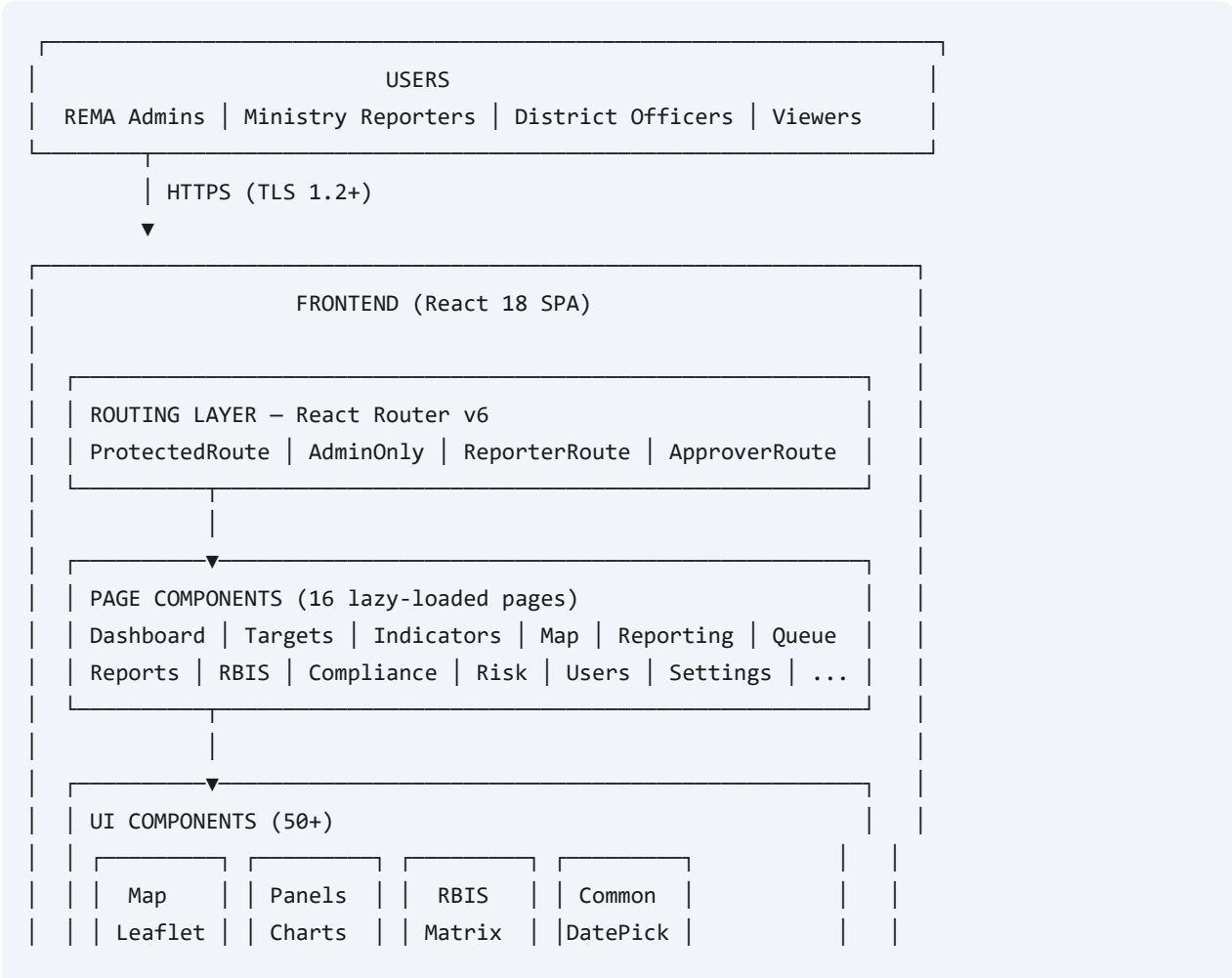
Supabase Edge Functions	Deno	Serverless functions (GBIF API proxy)
-------------------------	------	---------------------------------------

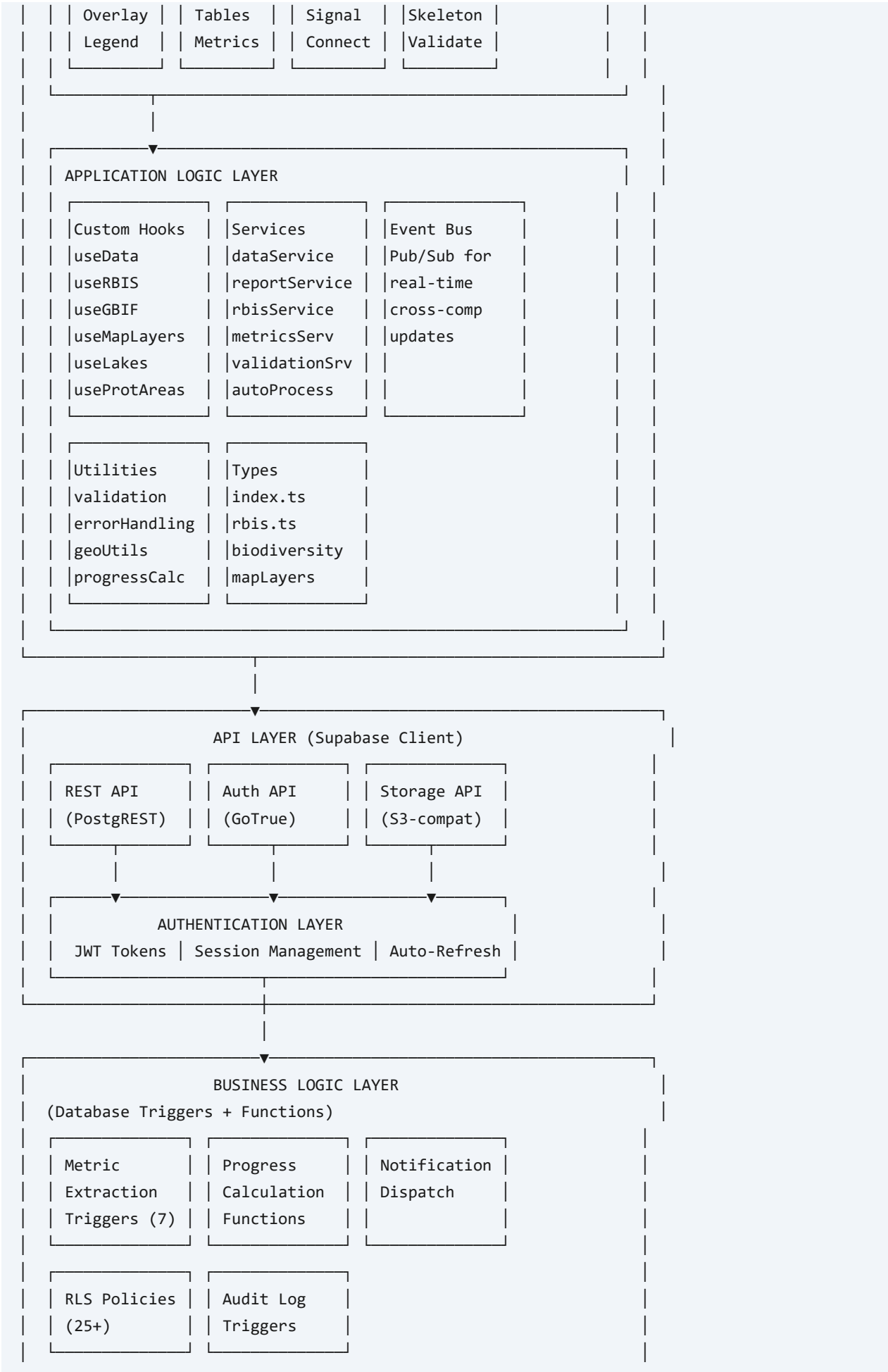
Infrastructure

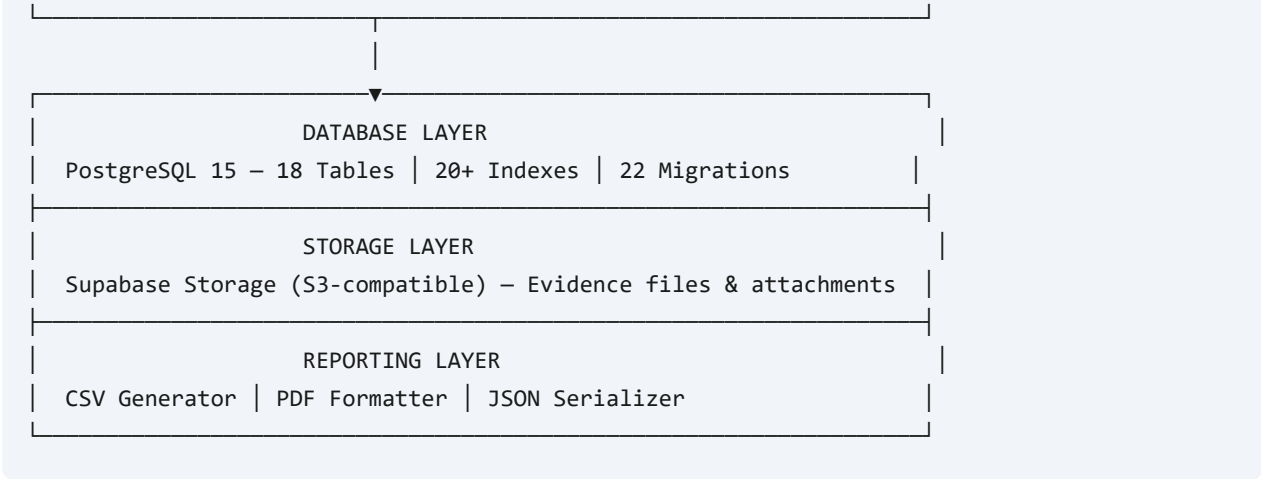
Component	Provider	Purpose
Frontend Hosting	Vercel	Global CDN with auto-deploy on git push
Backend Hosting	Supabase Cloud	Managed PostgreSQL, Auth, Storage, Realtime
DNS & SSL	Vercel	Automatic HTTPS certificate provisioning
CI/CD	Vercel	Git push → auto-build → deploy pipeline
Version Control	GitHub	Source code management
Backup	Supabase	Daily automated database backups

SECTION 13: COMPONENT DIAGRAM

13.1 Full Component Architecture





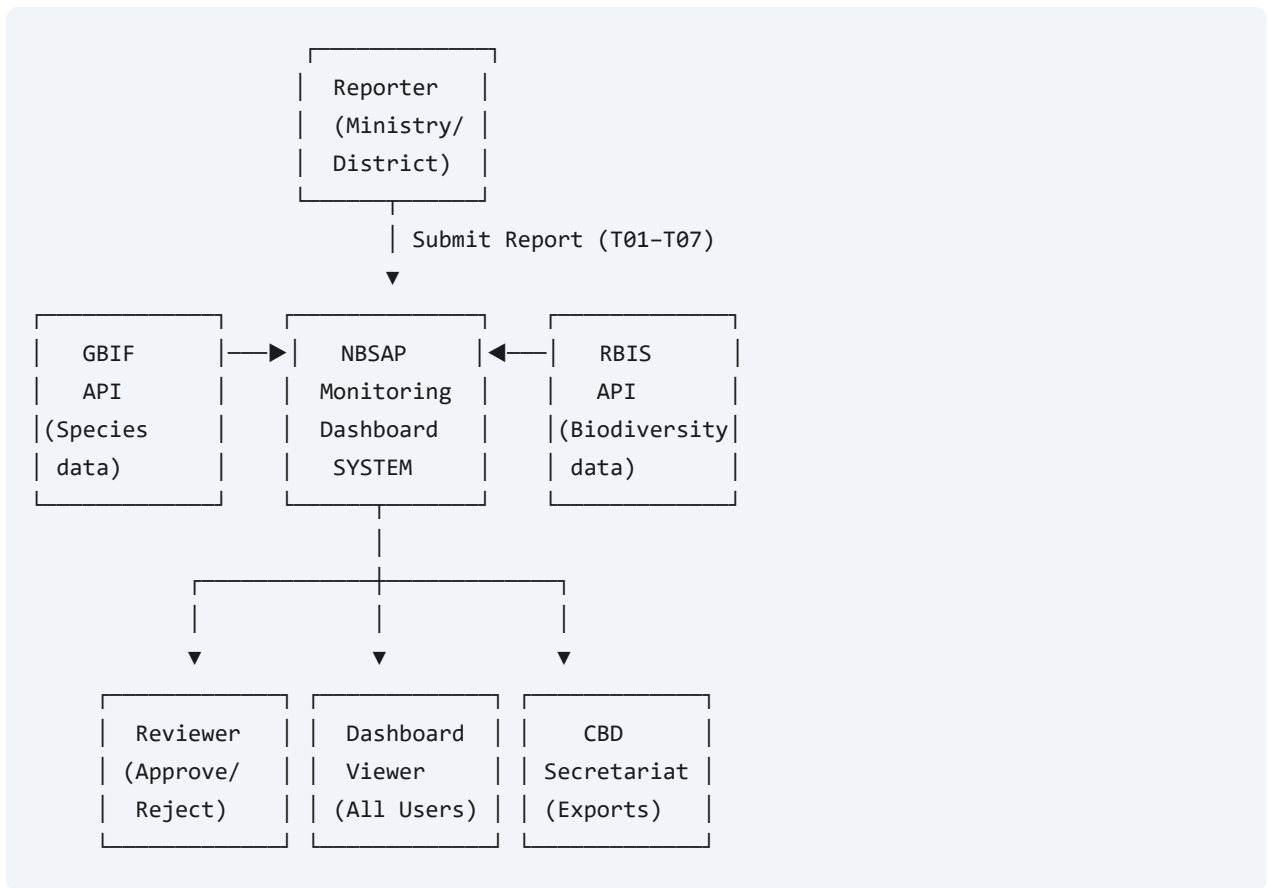


13.2 Component Explanations

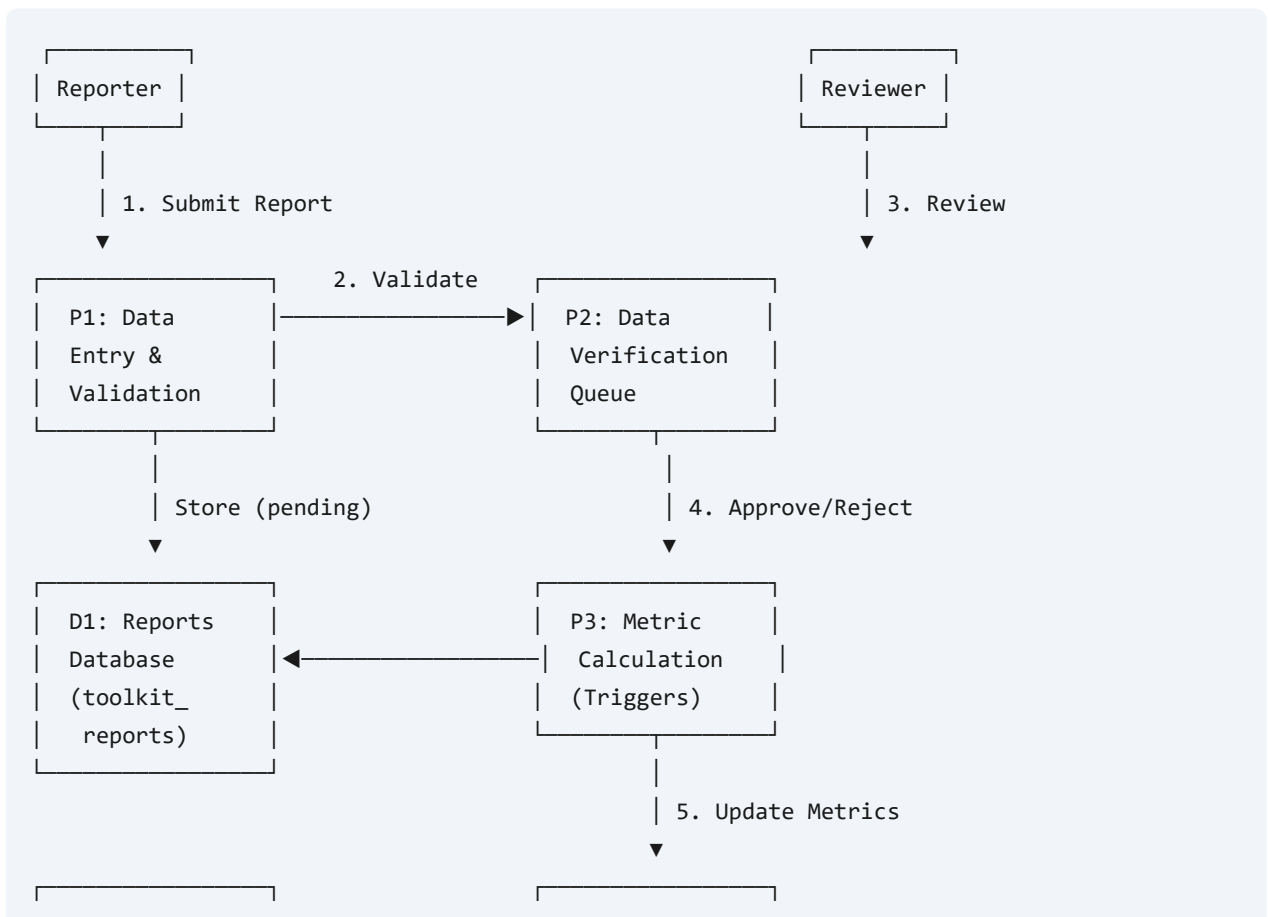
Component	Responsibility
Routing Layer	Enforces role-based page access; redirects unauthenticated users to login
Page Components	16 lazy-loaded pages; each represents a distinct functional area
UI Components	50+ reusable components: map overlays, data panels, forms, charts, shared elements
Custom Hooks	Encapsulate data fetching, state management, and side effects for reuse
Services	API client functions: CRUD operations, validation, metrics, RBIS integration
Event Bus	Typed pub/sub system for cross-component real-time updates
Utilities	Shared functions: validation, error handling, geographic calculations, progress computation
Types	TypeScript interfaces and enums ensuring type safety across the entire codebase
Supabase Client	JavaScript SDK connecting to Supabase REST, Auth, Storage, and Realtime APIs
Database Triggers	7 PostgreSQL triggers automating metrics, notifications, and audit logging
RLS Policies	25+ PostgreSQL policies enforcing data access at the database level
Storage Layer	S3-compatible storage for evidence documents and report attachments

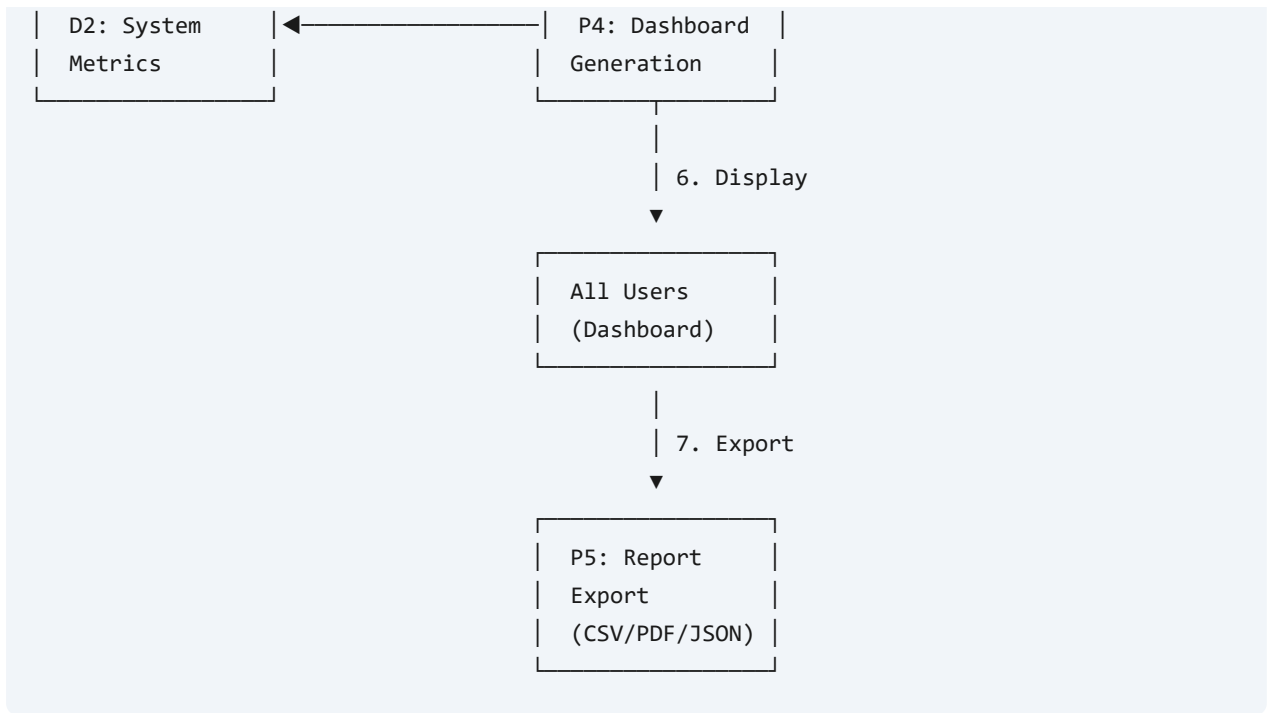
SECTION 14: DATA FLOW DIAGRAM (DFD)

14.1 Level 0 DFD — Context Diagram



14.2 Level 1 DFD — Major Processes





14.3 Level 2 DFD — Data Entry Process (P1)



- └─ 1.8 Reviewer Notified
 - └─ notifications: "New submission from [reporter]"

14.4 Level 2 DFD — Approval Process (P3)

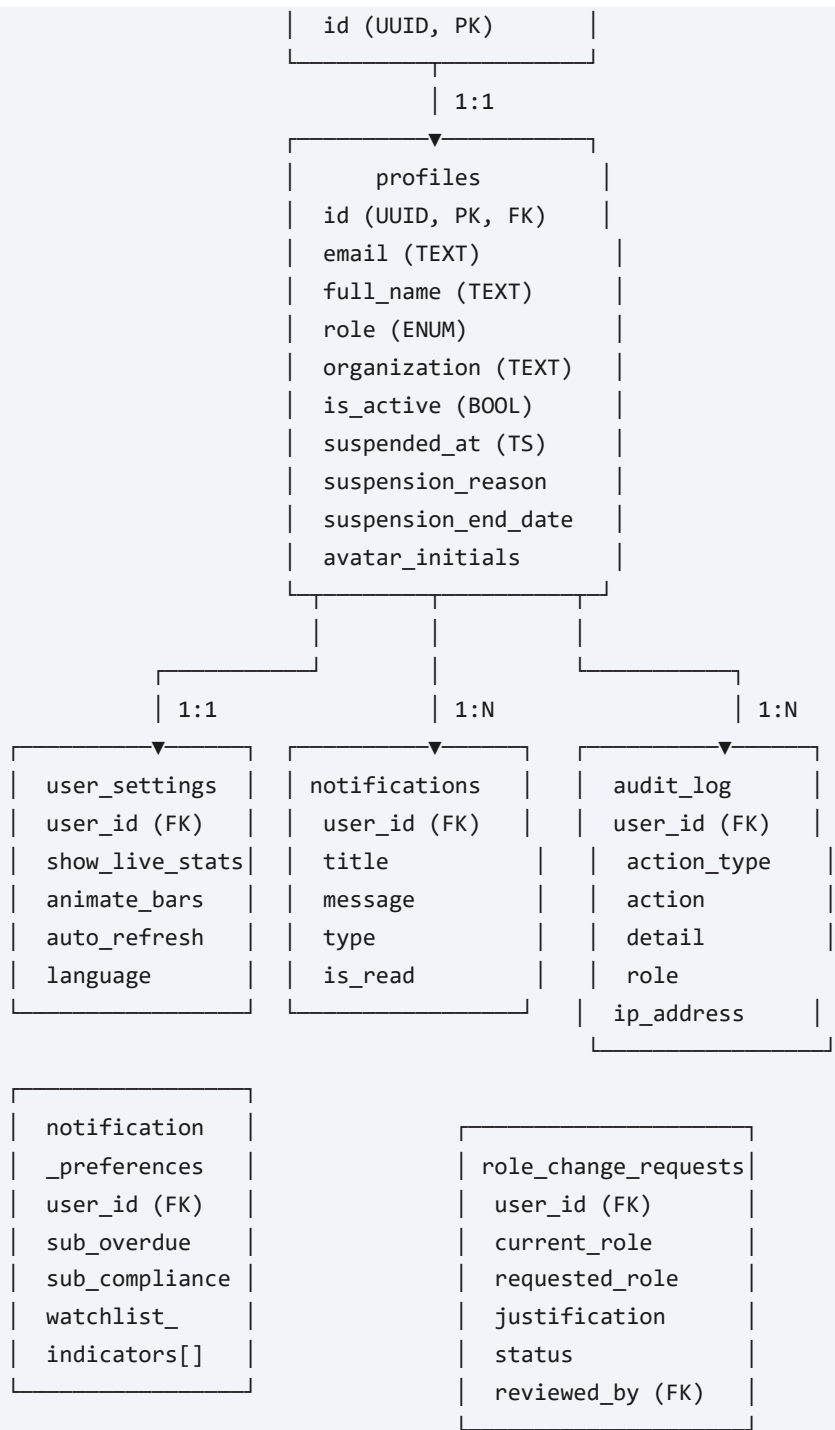
Reviewer approves report

- └─ 3.1 Update Report Status
 - └─ toolkit_reports.status = 'approved'
- └─ 3.2 Trigger: Extract Metrics
 - └─ Parse form_data JSONB for metric-relevant values
- └─ 3.3 Update System Metrics
 - └─ system_metrics: increment/update affected metrics
- └─ 3.4 Trigger: Weighted Progress
 - └─ Lookup tool_weights for this tool × target
 - └─ Calculate new target progress
 - └─ nbsap_targets.progress updated
- └─ 3.5 WebSocket Broadcast
 - └─ Supabase Realtime pushes event to all connected clients
- └─ 3.6 Dashboard Update
 - └─ Event bus emits 'target-progress-updated'
 - └─ Dashboard metrics refresh
 - └─ Progress bars re-render
- └─ 3.7 Reporter Notified
 - └─ "Your report has been approved"
- └─ 3.8 Audit Entry
 - └─ audit_log: action_type='approve'

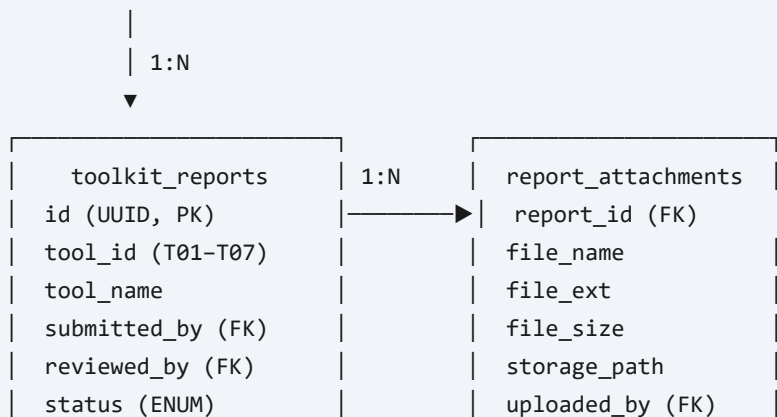
SECTION 15: ENTITY RELATIONSHIP DIAGRAM (ERD)

15.1 Complete ERD

```
┌──────────────────┐
│ auth.users        │
│ (Supabase)       │
└──────────────────┘
```



profiles.id (submitted_by / reviewed_by)



	form_data (JSONB)	
	attachments (JSONB)	
	district	
	institution	
	nbsap_target_id (FK)	
	submitted_at	

N:1

	nbsap_targets	
	id (INT, PK, 1-22)	
	goal (ENUM A-D)	
	title	
	description	
	baseline	
	headline_indicator	
	responsible_stk[]	
	progress (0-100)	

1:N

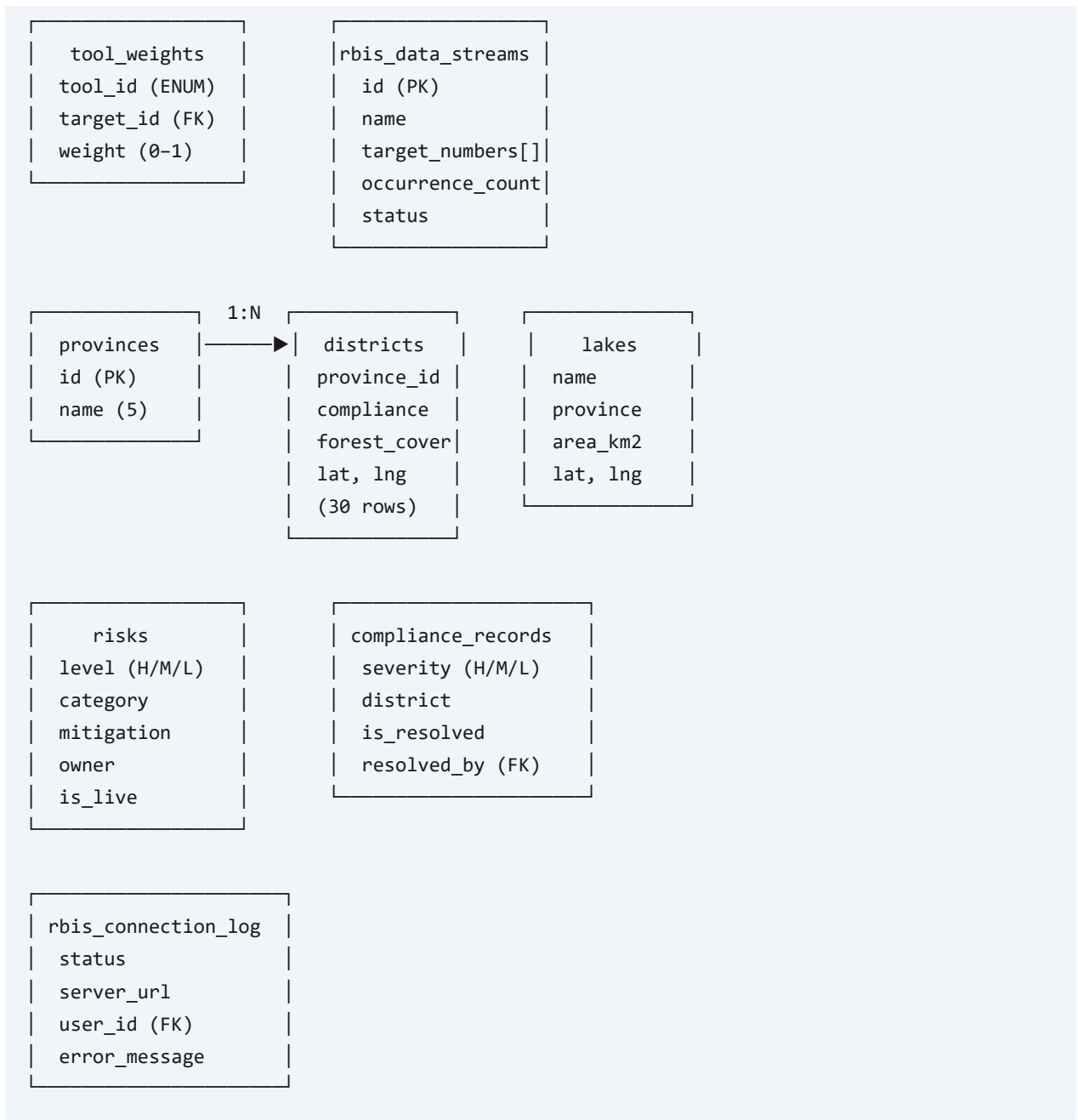
	indicators	
	id (SERIAL, PK)	
	nbsap_target_id(FK)	
	name	
	tier (ENUM)	
	progress (0-100)	
	status (ENUM)	
	baseline	
	current_value	
	final_target	
	responsible[]	

1:N

	rbis_linkages	
	indicator_id (FK)	
	data_stream_id	
	linkage_status	
	last_sync	

	nbsap_milestones	
	target_id (FK)	
	phase	
	milestone	
	status	

	system_metrics	
	metric_type	
	metric_value	
	metric_label	
	unit	
	last_updated	



15.2 Cardinality Summary

Relationship	Cardinality	Description
auth.users → profiles	1:1	Every auth user has exactly one profile
profiles → user_settings	1:1	One settings record per user
profiles → notification_preferences	1:1	One preferences record per user
profiles → toolkit_reports	1:N	A user can submit many reports
profiles → notifications	1:N	A user can receive many notifications
profiles → audit_log	1:N	A user generates many audit entries

nbsap_targets → indicators	1:N	A target has many indicators
nbsap_targets → toolkit_reports	1:N	A target receives many reports
nbsap_targets → nbsap_milestones	1:N	A target has many milestones
provinces → districts	1:N	A province contains many districts
toolkit_reports → report_attachments	1:N	A report can have many attachments
indicators → rbis_linkages	1:N	An indicator can link to many data streams

SECTION 16: DATABASE ARCHITECTURE

16.1 Database Design Principles

Principle	Implementation
Normalisation	3NF for core entities (users, targets, indicators, districts)
Flexible reporting	JSONB form_data column for dynamic tool-specific fields
Referential integrity	Foreign keys with CASCADE DELETE where appropriate
Security by default	Row-Level Security enabled on all 18 tables
Automation	7 triggers handle metrics, notifications, and audit logging
Performance	20+ indexes on frequently queried columns
Auditability	Immutable append-only audit_log table
Extensibility	22 sequential migrations for reproducible schema evolution

16.2 Table Summary

#	Table	Rows (Seed)	Purpose
1	profiles	Dynamic	User accounts extending auth.users
2	user_settings	Dynamic	Per-user dashboard preferences
3	notification_preferences	Dynamic	Per-user notification subscriptions
4	role_change_requests	Dynamic	Role change request workflow
5	nbsap_targets	22	National biodiversity targets
6	indicators	~79	Biodiversity indicators (4 tiers)

7	nbsap_milestones	~63	Implementation milestones 2025–2030
8	toolkit_reports	Dynamic	T01–T07 report submissions
9	report_attachments	Dynamic	Evidence files linked to reports
10	system_metrics	13	Automated system-wide metrics
11	tool_weights	~154 (7 tools × 22 targets)	Configurable tool contribution weights
12	audit_log	Dynamic (append-only)	Immutable action log
13	notifications	Dynamic	In-app notification delivery
14	risks	Dynamic	Biodiversity risk register
15	compliance_records	Dynamic	Compliance tracking
16	provinces	5	Rwanda's provinces
17	districts	30	Rwanda's districts
18	lakes	Dynamic	Inland water bodies
19	rbis_linkages	Dynamic	Indicator-to-stream mappings
20	rbis_data_streams	8	RBIS data stream definitions
21	rbis_connection_log	Dynamic	RBIS connection event audit

16.3 Key Relationships

Parent Table	Child Table	FK Column	ON DELETE
auth.users	profiles	profiles.id	CASCADE
profiles	user_settings	user_settings.user_id	CASCADE
profiles	notification_preferences	notification_preferences.user_id	CASCADE
profiles	notifications	notifications.user_id	CASCADE
profiles	toolkit_reports	toolkit_reports.submitted_by	SET NULL
profiles	audit_log	audit_log.user_id	SET NULL
nbsap_targets	indicators	indicators.nbsap_target_id	SET NULL
nbsap_targets	toolkit_reports	toolkit_reports.nbsap_target_id	SET NULL
toolkit_reports	report_attachments	report_attachments.report_id	CASCADE
provinces	districts	districts.province_id	SET NULL

indicators	rbis_linkages	rbis_linkages.indicator_id	CASCADE
------------	---------------	----------------------------	---------

16.4 Indexing Strategy

Strategy	Implementation
Primary keys	All tables have defined primary keys (UUID or SERIAL)
Foreign key indexes	Indexes on all FK columns for JOIN performance
Status filters	Indexes on <code>status</code> columns (toolkit_reports, indicators, districts, rbis_data_streams)
Temporal sorting	Descending indexes on <code>created_at</code> , <code>submitted_at</code> for recency queries
Composite indexes	<code>notifications(user_id, is_read)</code> for unread notification queries
Text search	<code>indicators.name</code> supports ILIKE search for indicator lookup

16.5 Database Constraints

Constraint Type	Examples
NOT NULL	profiles.email, indicators.name, toolkit_reports.tool_id
UNIQUE	districts.name, provinces.name
CHECK	progress BETWEEN 0 AND 100, compliance BETWEEN 0 AND 100
FOREIGN KEY	11+ FK relationships (see Section 16.3)
DEFAULT	status DEFAULT 'pending', is_active DEFAULT TRUE, created_at DEFAULT NOW()
GENERATED	profiles.avatar_initials GENERATED ALWAYS AS (UPPER(LEFT(full_name, 2)))
ENUM	8 PostgreSQL ENUM types enforcing valid values

SECTION 17: SECURITY ARCHITECTURE

17.1 Security Overview

LAYER 1: TRANSPORT SECURITY
└─ HTTPS/TLS 1.2+ on all connections
└─ HSTS headers enforced
└─ Vercel automatic SSL certificate management

LAYER 2: APPLICATION SECURITY

- └─ Content Security Policy (CSP)
- └─ X-Frame-Options: DENY (anti-clickjacking)
- └─ X-Content-Type-Options: nosniff
- └─ X-XSS-Protection: 1; mode=block
- └─ Client-side input validation (TypeScript)

LAYER 3: AUTHENTICATION

- └─ JWT-based sessions (Supabase Auth / GoTrue)
- └─ bcrypt password hashing (cost factor 10)
- └─ Auto-refresh tokens (seamless session extension)
- └─ Account status checks (active/suspended/deactivated)

LAYER 4: AUTHORIZATION (RBAC)

- └─ 6 user roles with scoped permissions
- └─ Client-side: ProtectedRoute components
- └─ Server-side: 25+ PostgreSQL RLS policies
- └─ Helper functions: get_user_role(), is_admin(), can_write(), is_role()
- └─ Cannot be bypassed via direct API access

LAYER 5: DATA SECURITY

- └─ Encryption at rest: AES-256 (Supabase managed)
- └─ Encryption in transit: TLS 1.2+
- └─ JSONB for flexible but typed data storage
- └─ Database constraints prevent invalid data states

LAYER 6: AUDIT & ACCOUNTABILITY

- └─ Immutable append-only audit_log table
- └─ Every significant action logged with user ID + timestamp
- └─ Verification gate prevents unreviewed data publication
- └─ Role change request audit trail

17.2 Authentication Flow

User enters email + password



Supabase Auth (GoTrue) validates credentials

- └─ bcrypt hash comparison
- └─ SUCCESS:
 - └─ JWT access token issued (1-hour expiry)
 - └─ Refresh token issued (long-lived)
 - └─ Profile fetched from profiles table
 - └─ Account status checked:


```
|
|   |   | is_active = false → BLOCKED (deactivated)
|   |   |   | suspended_at IS NOT NULL AND end_date > now → BLOCKED (suspended)
|   |   |   | suspended_at IS NOT NULL AND end_date < now → AUTO-REACTIVATE
|   |   |   | Active → PROCEED
|   |   |   | Redirect to dashboard
|
| FAILURE:
|   | Invalid credentials → error message
|   | Account not found → error message
```

17.3 Session Management

Aspect	Implementation
Token type	JWT (JSON Web Token)
Access token TTL	1 hour
Refresh mechanism	Automatic via Supabase JS client
Session storage	Browser memory + sessionStorage (2-layer cache, 5-min TTL)
Logout	Clear all tokens and session data; redirect to login
Stale detection	ProtectedRoute waits 50ms after auth load for session stabilisation

SECTION 18: INTEGRATIONS

18.1 Supabase Services

Service	Usage	Configuration
Supabase Database	PostgreSQL 15 — all application data	<code>VITE_SUPABASE_URL</code> , <code>VITE_SUPABASE_ANON_KEY</code>
Supabase Auth	User registration, login, JWT sessions, password reset	Supabase Auth configuration
Supabase Storage	S3-compatible storage for evidence files and report attachments	Bucket-level access policies
Supabase Realtime	WebSocket push for dashboard updates and notifications	Channel-based subscriptions
Supabase Edge Functions	GBIF API proxy (Deno runtime) — avoids CORS issues	<code>gbif-proxy</code> function deployed

18.2 GBIF API Integration

Attribute	Value
Provider	Global Biodiversity Information Facility
Base URL	<code>https://api.gbif.org/v1</code>
Proxy	Supabase Edge Function <code>gbif-proxy</code> (avoids CORS)
Authentication	None (public API)
Rate Limit	1 request/second (client-side enforced)
Data Used	Species occurrence records for Rwanda (country=RW)
Endpoints	<code>GET /occurrence/search?country=RW&limit=N&hasCoordinate=true</code>
Dashboard Usage	Map overlay (species points), RBIS metrics (total occurrences, recent records)
Failure Mode	Graceful degradation — dashboard functions without GBIF data

18.3 RBIS API Integration

Attribute	Value
Provider	Rwanda Biodiversity Information System (University of Rwanda)
Base URL	<code>https://rbis.ur.ac.rw/api/v1</code>
Authentication	Bearer token (<code>VITE_RBIS_API_TOKEN</code>)
Timeout	10,000ms
Cache Duration	300,000ms (5 minutes)
Data Streams	8 predefined streams linked to NBSAP indicators
Health Check	<code>GET /health</code> — monitors connection status
Failure Mode	Connection indicator turns red; dashboard uses local data only

18.4 GeoJSON Static Data

Layer	File	Content
District Boundaries	<code>public/rwanda-districts.geojson</code>	30 district polygons across 5 provinces
Protected Areas	<code>public/rwanda-protected-areas.geojson</code>	National parks, forest reserves, wetland reserves

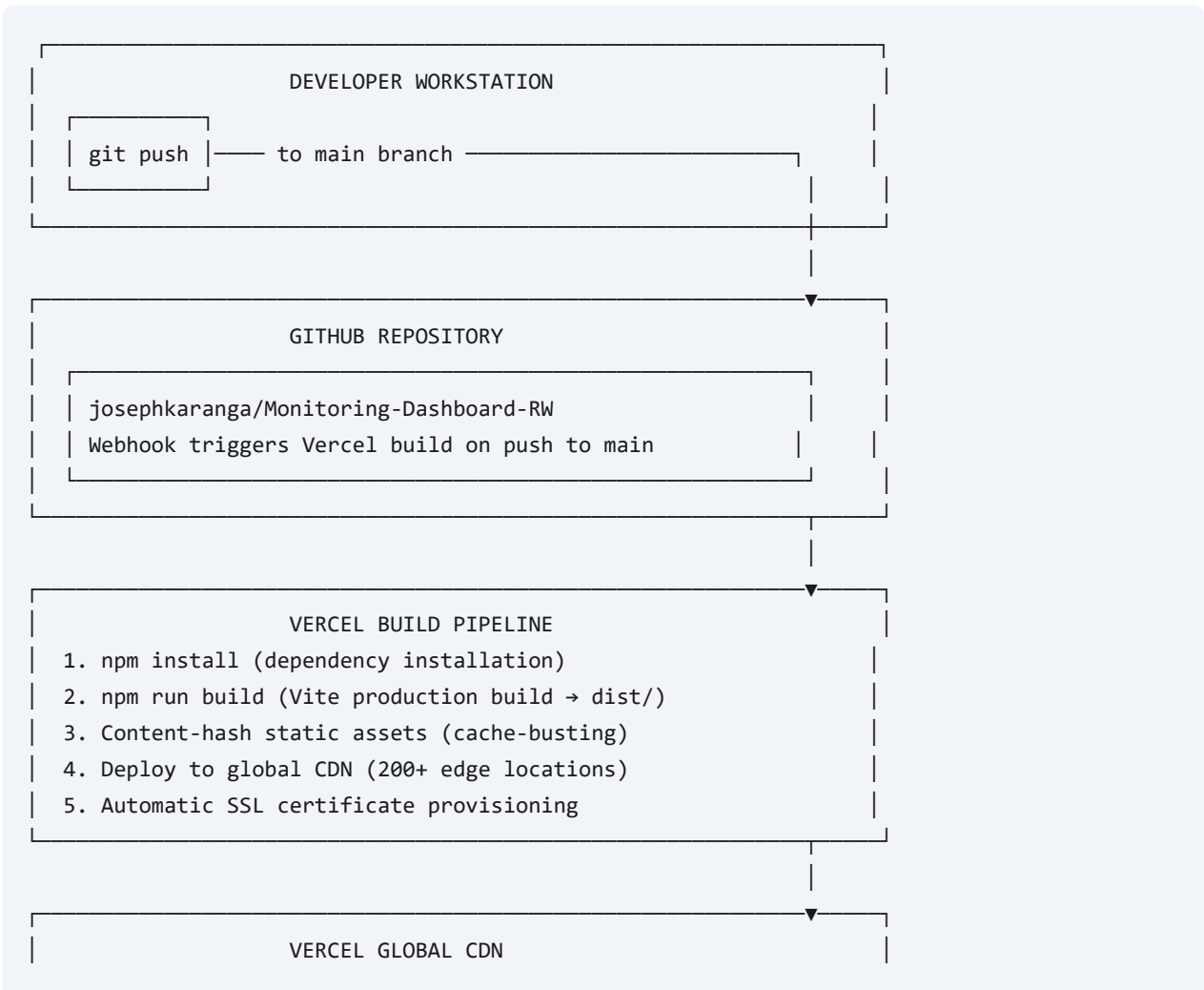
River Network	public/rwanda-rivers.geojson	Major river systems
---------------	------------------------------	---------------------

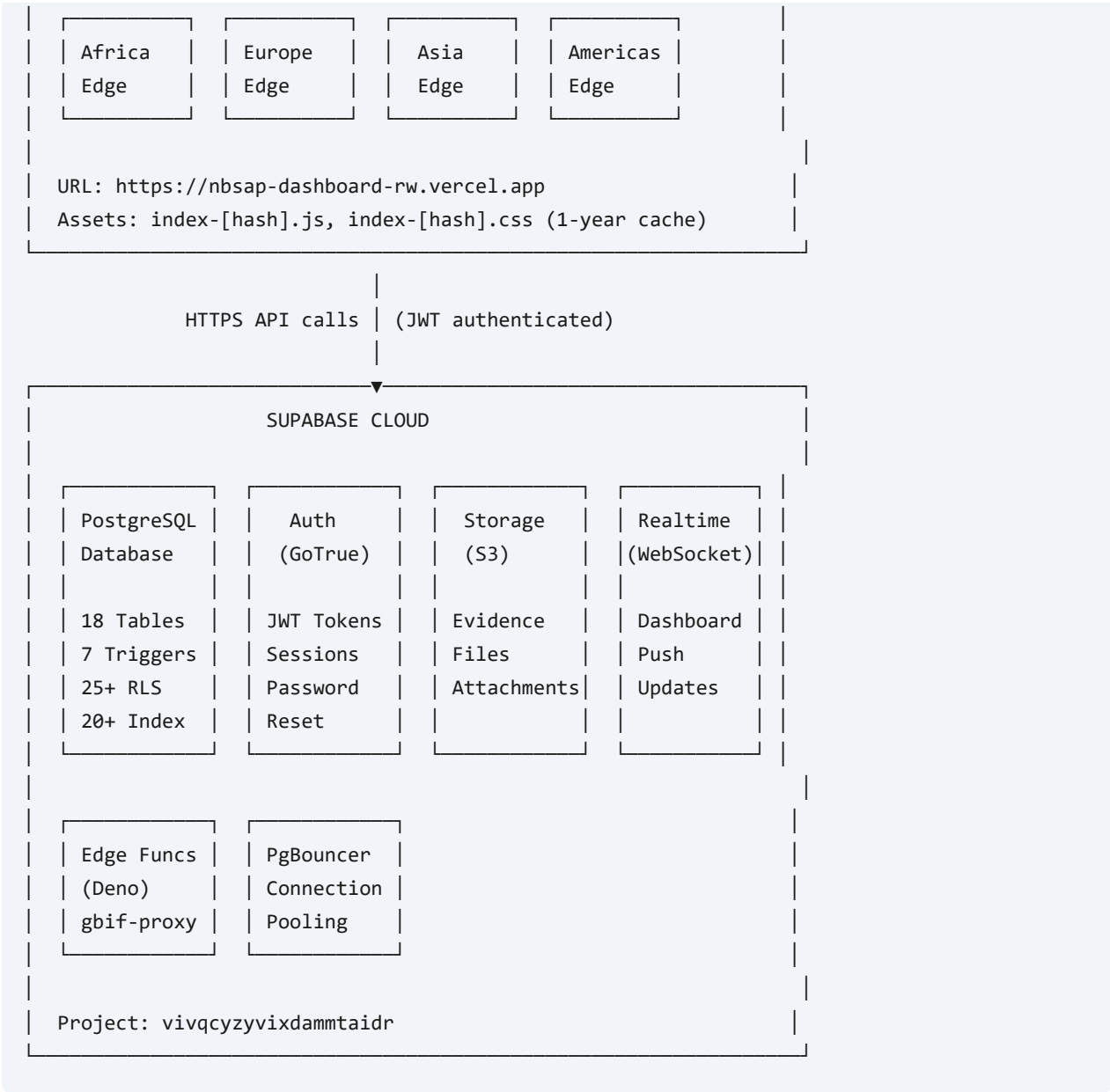
18.5 File Storage Integration

Attribute	Value
Provider	Supabase Storage (S3-compatible)
Purpose	Store evidence documents and report attachments
Access Control	Bucket-level policies; authenticated users can upload
File Types	PDF, DOC, DOCX, XLS, XLSX, JPG, PNG
Storage Path	Auto-generated unique paths per upload

SECTION 19: DEPLOYMENT ARCHITECTURE

19.1 Deployment Diagram





19.2 Environment Configuration

Environment	Purpose	URL
Development	Local development and testing	http://localhost:5173
Production	Live system serving all users	https://nbsap-dashboard-rw.vercel.app

Variable	Required	Description
VITE_SUPABASE_URL	Yes	Supabase project URL
VITE_SUPABASE_ANON_KEY	Yes	Supabase anonymous public key
VITE_APP_NAME	No	Application display name
VITE_APP_VERSION	No	Version string

VITE_RBIS_API_BASE_URL	No	RBIS API base URL
VITE_RBIS_API_TOKEN	No	RBIS API authentication token
VITE_ENABLE_RBIS_INTEGRATION	No	Enable/disable RBIS
VITE_ENABLE_REALTIME	No	Enable/disable WebSocket

19.3 Estimated Annual Cost

Service	Tier	Monthly Cost	Annual Cost
Vercel	Hobby / Pro	\$0 – \$20	\$0 – \$240
Supabase	Free / Pro	\$0 – \$25	\$0 – \$300
Domain (optional)	Standard	\$1	\$10 – \$15
Total			\$0 – \$555

SECTION 20: APPENDICES

Appendix A: Glossary

Term	Definition
NBSAP	National Biodiversity Strategy and Action Plan
KM-GBF	Kunming-Montreal Global Biodiversity Framework
CBD	Convention on Biological Diversity
REMA	Rwanda Environment Management Authority
GBIF	Global Biodiversity Information Facility
RBIS	Rwanda Biodiversity Information System
RLS	Row-Level Security (PostgreSQL feature)
JWT	JSON Web Token
BaaS	Backend-as-a-Service
SPA	Single-Page Application
CDN	Content Delivery Network
GIS	Geographic Information System
EIA	Environmental Impact Assessment

HWC	Human-Wildlife Conflict
ABS	Access and Benefit Sharing
IAS	Invasive Alien Species
NbS	Nature-based Solutions
TRSP	Tourism Revenue Sharing Program
PA	Protected Area
OECM	Other Effective area-based Conservation Measure

Appendix B: Acronyms — Government Institutions

Acronym	Full Name
REMA	Rwanda Environment Management Authority
MoE	Ministry of Environment
MINECOFIN	Ministry of Finance and Economic Planning
MINAGRI	Ministry of Agriculture and Animal Resources
MININFRA	Ministry of Infrastructure
MINICOM	Ministry of Trade and Industry
MINEDUC	Ministry of Education
RDB	Rwanda Development Board
RFA	Rwanda Forestry Authority
RAB	Rwanda Agriculture and Animal Resources Development Board
RNRA	Rwanda Natural Resources Authority
WASAC	Water and Sanitation Corporation
BNR	National Bank of Rwanda
PSF	Private Sector Federation
RBS	Rwanda Bureau of Standards
NISR	National Institute of Statistics of Rwanda

Appendix C: Assumptions

#	Assumption	Rationale
---	------------	-----------

1	District officers have internet access	Rwanda's broadband coverage exceeds 95% of urban areas
2	REMA will designate at least 2 system administrators	Ongoing system management requires institutional commitment
3	GBIF API remains publicly accessible	GBIF is multilateral infrastructure with long-term funding
4	Supabase maintains service availability	Supabase provides 99.9% uptime SLA
5	Ministry reporters are trained on the 7 reporting tools	Training is part of system deployment
6	Data collection follows quarterly/annual reporting cycles	Aligned with Rwanda's existing M&E calendar
7	Evidence documents are available in digital format	Increasing digitisation of field operations

Appendix D: Dependencies

Dependency	Type	Risk
Supabase Cloud	Infrastructure	Medium — single provider for database, auth, storage
Vercel CDN	Infrastructure	Low — easy to migrate to Netlify or self-host
GBIF API	External data	Low — system functions without GBIF data
RBIS API	External data	Low — system functions without RBIS data
React ecosystem	Technology	Low — LTS; large community; stable
PostgreSQL	Technology	Very Low — industry standard; managed by Supabase
Modern browsers	Client	Low — Rwanda's browser adoption skews modern

Appendix E: Risks

#	Risk	Likelihood	Impact	Mitigation
1	Supabase free tier limits exceeded	Medium	Medium	Monitor usage; upgrade to Pro tier (\$25/month)
2	Key personnel leave REMA	Medium	High	Document all processes; train multiple administrators
3	GBIF API becomes unavailable	Low	Low	System functions without GBIF; cached data shown

4	Security breach attempt	Medium	High	RLS policies, HTTPS, CSP headers, audit logging, JWT auth
5	Data loss	Low	Critical	Daily automated Supabase backups; git-versioned migrations
6	Internet connectivity disruption	Medium	Medium	System designed for low-bandwidth; CDN caching reduces data transfer
7	Browser compatibility issues	Low	Low	Standard React/TypeScript; modern browser requirement documented

Appendix F: Recommendations

#	Recommendation	Priority	Rationale
1	Upgrade Supabase to Pro tier before user base exceeds 100	High	Ensures database, storage, and backup capacity
2	Implement multi-language support (Kinyarwanda, French)	Medium	Improves accessibility for district-level officers
3	Develop mobile application (React Native)	Medium	Enables offline field data collection
4	Automate CBD National Report generation	High	Reduces manual effort for periodic CBD reporting
5	Add SMS notifications for overdue reports	Medium	Reaches users with limited email access
6	Conduct quarterly security audits	High	Maintains security posture as system evolves
7	Establish data governance committee	High	Institutional oversight of data quality and access policies
8	Integrate with Rwanda's e-Government portal	Low	Long-term alignment with national digital infrastructure

Prepared by: NBSAP Dashboard Development Team

Document Reference: NBSAP-COMP-2026-001

Rwanda Environment Management Authority (REMA)

System: Rwanda NBSAP Monitoring Dashboard v1.0.0

Date: June 2026